

1. Ce va afișa la rulare următorul program?.

```
public delegate void Del(int n);
public class TestDelegate
{
    public void VerificareParitate(int n)
    {
        if (n % 2 == 0)
            Console.WriteLine("Numar par");
        else
            Console.WriteLine("Numar impar");
    }

    public void PatratNumar(int n)
    {
        Console.WriteLine("Patrat numar: {0}", n*n);
    }
}

class Exemplu
{
    public static void Main()
    {
        TestDelegate obj = new TestDelegate();
        Del delObj = new Del(obj.VerificareParitate);
        delObj += new Del(obj.PatratNumar);
        delObj(-3);
    }
}
```

programul va rula fără probleme însă nu va afișa nimic

Numar impar

Numar impar
Patrat numar: 9

nu va afișa nimic deoarece va da eroare la rulare

Patrat numar: 9

1. Declararea și Instanțierea Delegatului:

- În metoda Main, se creează o instanță a clasei TestDelegate.
- Apoi, se creează o instanță a delegatului Del, numită delObj, care este asociată cu metoda obj.VerificareParitate.

2. Crearea unui Delegat Multicast:

- Instrucțiunea delObj += new Del(obj.PatratNumar); adaugă metoda obj.PatratNumar la lista de metode care vor fi invocate de delObj.
- Acest lucru transformă delObj într-un delegat *multicast*, ceea ce înseamnă că la apelarea lui, se vor executa în ordine toate metodele asociate.

3. Invocarea Delegatului:

- Apelul delObj(-3); va executa metodele în ordinea în care au fost adăugate:
 1. **VerificareParitate(-3):** Deoarece $-3 \% 2$ este diferit de 0, condiția if este falsă, iar programul va executa ramura else, afișând la consolă "Numar impar".
 2. **PatratNumar(-3):** Această metodă va calcula $(-3) * (-3)$, care este 9, și va afișa la consolă "Patrat numar: 9".

2. Ce este covarianța?

tipul returnat e mai derivat decat cel definit de delegat

tipul returnat e mai puțin derivat decat cel definit de delegat

tipul parametrilor poate fi mai derivat decat cele definite de delegati

tipul parametrilor poate fi mai puțin derivat decat cele definite de delegati

În documentul "03 Programare (1).pdf", la secțiunea "3.13.1. Covarianță și contravarianță", se specifică următoarele:

- "Covarianța permite unei metode să aibă **tipul returnat mai derivat decât cel definit de delegat.**"

3. Ce este contravarianța?

tipul returnat e mai derivat decat cel definit de delegat

tipul returnat e mai puțin derivat decat cel definit de delegat

tipul parametrilor poate fi mai derivat decat cele definite de delegati

tipul parametrilor poate fi mai puțin derivat decat cele definite de delegati

- "Contravarianța permite unei metode să aibă **tipuri de parametri mai puțin derivate decât cele din tipul delegat.**"

4. Ce raspuns este adevarat despre WPF?

designul interfetei se face prin intermediul unui fisier XML

interfața este definita pentru o anumita rezoluție

designul interfetei se face numai prin code-behind

În documentul "08 WPF.pdf" se menționează că WPF folosește XAML (Extensible Application Markup Language) pentru a defini interfața cu utilizatorul. Același document clarifică faptul că "XAML este un limbaj bazat pe XML care poate fi folosit pentru a crea o interfață cu utilizatorul în WPF".

5. Ce returneaza urmatorul cod pentru apelul Metoda(1234) ?

```
public double Metoda(int n)
{
    if (n == 0)
        return 0;
    else
        return n % 10 + 10 * Metoda(n / 10);
}
```

1234

9

4321

24

1. **Metoda(1234)** returnează $1234 \% 10 + 10 * \text{Metoda}(1234 / 10)$ adică $4 + 10 * \text{Metoda}(123)$
2. **Metoda(123)** returnează $123 \% 10 + 10 * \text{Metoda}(123 / 10)$ adică $3 + 10 * \text{Metoda}(12)$
3. **Metoda(12)** returnează $12 \% 10 + 10 * \text{Metoda}(12 / 10)$ adică $2 + 10 * \text{Metoda}(1)$
4. **Metoda(1)** returnează $1 \% 10 + 10 * \text{Metoda}(1 / 10)$ adică $1 + 10 * \text{Metoda}(0)$
5. **Metoda(0)** este cazul de bază și returnează 0.

Acum, valorile se calculează înapoi, în ordinea inversă a apelurilor:

- La pasul 4, **Metoda(1)** returnează $1 + 10 * 0 = 1$.
- La pasul 3, **Metoda(12)** returnează $2 + 10 * 1 = 12$.
- La pasul 2, **Metoda(123)** returnează $3 + 10 * 12 = 123$.
- La pasul 1, **Metoda(1234)** returnează $4 + 10 * 123 = 1234$.

6. Ce va afisa urmatoarea secventa de cod?

```
checked
{
    int i = int.MaxValue;
    i++;
    Console.WriteLine(i);
}
```

2,147,483,647

0

va da eroare de overflow

nu va afisa nimic

- `int.MaxValue` este 2,147,483,647 (valoarea maximă pe care o poate stoca un `int` în C#).
- `i++` înseamnă `i = i + 1`, dar `int.MaxValue + 1` depășește limita tipului `int`.
- `checked` este un bloc care activează verificarea explicită a overflow-ului pentru operațiile aritmetice.

Dacă apare overflow, se aruncă o excepție `System.OverflowException`.

Fără `checked`, comportamentul implicit ar fi "wrap-around" (adică `int.MaxValue + 1 = int.MinValue = -2,147,483,648`).

Dar aici, `checked` forțează verificarea, deci codul nu va ajunge la `Console.WriteLine` și va arunca eroare.

7.Cum se poate accesa lista de fonturi instalate in sistemul de operare?

`FontLoaderList`

`FontCollection`

`InstalledFontCollection`

În documentul "06 Grafica (1).pdf", secțiunea "System.Drawing.Text", se specifică următoarele:

- "Pentru listarea fonturilor sistemului, un program trebuie să creeze o instanță a clasei **InstalledFontCollection** și să utilizeze metoda `Families()` pentru obținerea unui șir de obiecte de tip `FontFamily` instalate."

Clasa `FontCollection` este menționată ca fiind o clasă de bază pentru `InstalledFontCollection` și `PrivateFontCollection`.

`FontLoaderList` nu este menționată în documentele furnizate.

8.Ce va afisa urmatoarea secventa de cod?

```
for (int i = 0; i < 5; i += 2)
{
    for (int j = 0; j < 5; j += 2)
    {
        if (i == j) break;
        Console.WriteLine("{0} ", j++);
    }
}
```

0 3 0 3

0 2 4 0 2 4

0 0 2 0 0 4

1. Bucla exterioară (i = 0):

- Se intră în bucla interioară (`j` începe de la 0).
- Condiția `if (i == j)` (adică `0 == 0`) este adevărată.
- Instrucțiunea `break;` este executată, ceea ce oprește imediat **bucla interioară**.
- Nu se afișează nimic.

2. Bucla exterioară (i = 2):

Se intră în bucla interioară (j începe din nou de la 0).

- **j = 0:**
 - Condiția `if (i == j) (2 == 0)` este falsă.
 - Se execută `Console.WriteLine("{0} ", j++)`; . Se afișează valoarea curentă a lui j (0), apoi j este incrementat la 1.
 - La finalul iterației, se execută `j += 2`, deci j devine $1 + 2 = 3$.
- **j = 3:**
 - Condiția `if (i == j) (2 == 3)` este falsă.
 - Se execută `Console.WriteLine("{0} ", j++)`; . Se afișează valoarea curentă a lui j (3), apoi j este incrementat la 4.
 - La finalul iterației, se execută `j += 2`, deci j devine $4 + 2 = 6$.
- **j = 6:**
 - Condiția buclei `j < 5` este acum falsă, deci bucla interioară se oprește.
- Până acum, s-a afișat: 0 3

3. Bucla exterioară (i = 4):

- Se intră în bucla interioară (j începe din nou de la 0).
 - **j = 0:**
 - Condiția `if (i == j) (4 == 0)` este falsă.
 - Se afișează 0, j devine 1, apoi `j += 2` îl face 3.
 - **j = 3:**
 - Condiția `if (i == j) (4 == 3)` este falsă.
 - Se afișează 3, j devine 4, apoi `j += 2` îl face 6.
 - **j = 6:**
 - Condiția buclei `j < 5` este falsă, deci bucla interioară se oprește.
- Până acum, s-a afișat: 0 3 0 3

4. Bucla exterioară (i = 6):

- Condiția buclei exterioare `i < 5` este acum falsă, deci programul se încheie.

Rezultatul final afișat la consolă este 0 3 0 3.

9.Ce face urmatoarea clasa?

```
public class MyCollection
{
    private string[] date = new string[10];

    public string this[int index]
    {
        get { return date[index]; }
        set { date[index] = value; }
    }
}
```

defineste o colectie de tip indexator

are eroare de sintaza

creaza o metoda de extensie pentru tipul string

Clasa `MyCollection` folosește o construcție specifică limbajului `C#` numită **indexator**.

În documentul "03 Programare (1).pdf", secțiunea "3.8. Indexatori", se menționează:

- "Indexatorii reprezintă o convenție de sintaxă care permit crearea unei clase, structuri sau interfețe pe care aplicațiile client le pot accesa întocmai ca pe un **vector**."
- "Indexatorii sunt cel mai frecvent implementați în tipurile ale căror scop principal este să încapsuleze o **colecție internă sau un vector**."

Codul din exemplu face exact acest lucru:

1. Încapsulează un vector privat (`private string[] date`).
2. Folosește sintaxa `public string this[int index]` pentru a declara un **indexator**.
3. Acest indexator permite accesarea elementelor vectorului privat `date` direct printr-o instanță a clasei `MyCollection`, ca și cum ar fi un vector (de ex. `myCollectionObject[i]`).

Celelalte opțiuni sunt incorecte:

- **are eroare de sintaxă:** Sintaxa este corectă pentru definirea unui indexator în `C#`.
- **creaza o metoda de extensie pentru tipul string:** O metodă de extensie este definită într-o clasă statică și are o sintaxă diferită (de ex. `public static int WordCount(this string s)`).

10. Ce va afișa următoarea secvență de cod?

```
public class Program
{
    enum Color
    {
        Rosu = 1,
        Verde = 2,
        Albastru = 3
    }

    static void Main(string[] args)
    {
        Color c = Color.Rosu | Color.Albastru;
        Console.WriteLine(c);
    }
}
```

Albastru

eroare

5

Rosu Albastru

1. **Operația pe Enumerare:** Codul efectuează o operație binară OR (`|`) între membrii enumerării `Color`. Aceste operații se execută pe valorile întregi asociate fiecărui membru.
 - o `Color.Rosu` are valoarea 1 (reprezentare binară: 01).
 - o `Color.Albastru` are valoarea 3 (reprezentare binară: 11).
2. **Calculul Binar:** Operația binară OR între 1 și 3 este:
 3. 01 (Rosu = 1)
 4. | 11 (Albastru = 3)
 5. ----
 6. 11 (Rezultat = 3)

Rezultatul operației este valoarea întreagă 3.

Notă: Enumerarea `Color` nu folosește atributul `[Flags]`. Chiar dacă l-ar folosi, pentru ca afișarea să fie "Rosu, Albastru", membrii ar trebui să aibă valori care sunt puteri ale lui 2 (de ex., `Rosu = 1`, `Albastru = 4`), iar rezultatul operației `OR` (`1 | 4 = 5`) nu ar corespunde niciunui membru individual, fiind astfel afișată o listă. În cazul de față, rezultatul (3) corespunde exact membrului `Albastru`.

11.Ce va afisa urmatoarea secventa de cod?

```
double a = 5.9f;
float b = (float)a;
Console.WriteLine(b);
```

6
5.9
5.9f
true

1. `double a = 5.9f;`:

- Valoarea `5.9f` este o constantă literală de tip `float`, conform documentului "03 Programare (1).pdf".
- Această valoare `float` este atribuită unei variabile de tip `double`. Aceasta este o conversie de tip *widening* (de la un tip mai mic la unul mai mare), care se realizează implicit și fără pierderi de informații => Variabila `a` va conține valoarea `5.9`.

2. `float b = (float)a;`:

- Valoarea `double` din `a` este convertită explicit (printr-un *cast*) la tipul `float`. Aceasta este o conversie de tip *narrowing* (de la un tip mai mare la unul mai mic).
- Deoarece valoarea originală (`5.9f`) provine dintr-un `float`, conversia înapoi la `float` nu va produce pierderi de precizie în acest caz. Variabila `b` va conține valoarea `5.9`

3. `Console.WriteLine(b);`:

- Această instrucțiune afișează la consolă reprezentarea sub formă de șir de caractere a valorii stocate în variabila `b`, care este `5.9`.

Opțiunile "6" (nu are loc nicio rotunjire), "5.9f" (sufixul `f` este folosit doar în cod, nu la afișare) și "true" (se afișează o valoare numerică, nu una booleană) sunt incorecte.

12.Ce returneaza urmatoare secventa de cod?

```
var a = int.TryParse("128", out x);
Console.WriteLine(a);
```

0
128
eroare

Motivul este că variabila `x` este folosită ca parametru `out` în metoda `int.TryParse`, dar **nu este declarată** nicăieri înainte de utilizare. În C#, o variabilă trebuie declarată înainte de a putea fi folosită, chiar și ca parametru `out`.

```
int x; var a = int.TryParse("128", out x); Console.WriteLine(a); || ("128", out int x);
```

Dacă codul ar fi fost scris corect în oricare dintre aceste variante, `int.TryParse` ar fi returnat `true` (deoarece "128" este un număr valid), iar programul ar fi afișat **True**.

13. Ce va afișa următorul program?

```
public static void Metoda1(ref int a)
{
    a += a;
}

public static void Metoda2(out int a)
{
    a = 20;
}

public static void Main()
{
    int i;
    int j = 20;
    Metoda1(ref i);
    Console.WriteLine(i);
    Metoda2(out j);
    Console.WriteLine(j);
}
```

3 20

eroare parametrul i este neinitializat

6 20

eroare parametrul out j este initializat

Conform documentului "03 Programare (1).pdf", secțiunea "3.6.1. Metode cu parametri out și ref", există reguli stricte pentru utilizarea parametrilor de tip `ref`:

- **Regula pentru `ref`:** Materialul de curs specifică: "Apelantul este obligat să se asigure că orice variabile transmise cu `ref` conțin o valoare înainte de a efectua apelul".

În secvența de cod din `Main`, variabila `i` este declarată (`int i;`), dar nu i se atribuie nicio valoare inițială. Deoarece variabila `i` este neinițializată, încercarea de a o pasa ca parametru `ref` către `Metoda1` încalcă această regulă, iar compilatorul va semnaliza o eroare.

Cealaltă opțiune de eroare, "eroare parametrul out j este initializat", este incorectă. Este permisă pasarea unei variabile inițializate ca parametru `out`, însă valoarea ei inițială va fi ignorată, deoarece metoda este obligată să îi atribuie o nouă valoare.

14. Ce este un delegat?

un tip de date care conține referința spre o metodă

o metodă statică

o metodă ce răspunde la evenimente

un handler de evenimente

În documentul "03 Programare (1).pdf", secțiunea "3.7. Delegați", se oferă următoarea definiție:

- "Un delegat este un tip care definește o semnătură a unei metode. O variabilă de tip delegat poate reține o **referință la o metodă** cu o semnătură compatibilă."

Deși delegații sunt folosiți frecvent pentru a implementa handleri de evenimente, definiția lor fundamentală este aceea de a fi un tip de date ce acționează ca un "pointer" sigur la o metodă.

15.Cum se poate lega metoda `MyMethod` de un buton in WPF in XML?

```
Click="MyMethod"  
Click+=MyMethod  
Click+=MyMethod()
```

În documentul "08 WPF.pdf", secțiunea "8.1.2. Code-behind", este prezentat un exemplu care ilustrează exact acest mecanism.

Codul XAML relevant din document este:

```
<Button Name="button1" Click="button1_Click">Click Me!</Button>
```

- `Click+=MyMethod` reprezintă sintaxa C# pentru adăugarea unui handler de eveniment în fișierul code-behind, nu sintaxa XAML.
- `Click+=MyMethod()` este o sintaxă invalidă în acest context.

16.Ce va afisa urmatoarea secventa de cod?

```
byte b = (byte)(175 + 264);  
Console.WriteLine("{0}", b);  
183  
439  
220
```

1. Calculul $175 + 264 = 439$ (rezultatul este un int).
2. Conversia la byte: Tipul byte poate stoca valori între 0 și 255.

Dacă valoarea depășește 255, se aplică "trunchierea" (se ia doar partea inferioară a valorii, folosind operația mod 256).

$439 \% 256 = 183$ (deoarece $256 \times 1 = 256$, iar $439 - 256 = 183$).

17.Ce va afisa urmatoarea secventa de cod?

```
Console.WriteLine(sizeof(long) + sizeof(decimal));  
12  
24  
eroare  
36
```

Conform tabelului 3.5 din documentul "03 Programare (1).pdf" (pagina 42), dimensiunile acestor tipuri de date sunt:

- `long`: 8 octeți (bytes)
- `decimal`: 16 octeți

Prin urmare, calculul efectuat este $8 + 16$, ceea ce rezultă în 24.

18.Ce va face urmatoarea secventa de cod?

```
var vector = new[] { 1, "two", 3 };  
creeaza un vector de stringuri  
creeaza un vector de numere  
creaza un vector mixt de stringuri si numere  
eroare
```

În documentul "03 Programare (1).pdf", secțiunea "3.4.2. Vectori locali cu tipul dedus", se explică regulile pentru crearea vectorilor cu tipul dedus (folosind `var` și `new[]`).

- Regula principală este: "elementele din lista de inițializare **trebuie să fie de același tip**".

19. Ce va afișa la rulare următoarea secvență de cod?

```
class Clasa1
{
    public virtual void Metoda()
    {
        Console.WriteLine("Metoda1");
    }
}

class Clasa2 : Clasa1
{
    public override void Metoda()
    {
        Console.WriteLine("Metoda2");
    }
}

Clasa2 obj2 = new Clasa2();
obj2.Metoda();
Console.WriteLine(",");
Clasa1 obj1 = (Clasa1)obj2;
obj1.Metoda();
```

Metoda2, Metoda2

Metoda1, Metoda2

Metoda2, Metoda1

1. **Clasa2 obj2 = new Clasa2();** Se creează o instanță a clasei Clasa2.
2. **obj2.Metoda();** Se apelează metoda Metoda() pe obiectul obj2. Deoarece obj2 este de tip Clasa2, se execută metoda suprascrisă (cu override) din Clasa2, afișând **"Metoda2"**.
3. **Console.WriteLine(",");** Se afișează o virgulă.
4. **Clasa1 obj1 = (Clasa1)obj2;** Obiectul obj2 (care este o instanță de Clasa2) este convertit (upcast) la tipul clasei de bază, Clasa1. Acum, variabila obj1 este de tipul Clasa1, dar ea *continuă să facă referire la același obiect* care este, în realitate, o instanță de Clasa2.
5. **obj1.Metoda();** Se apelează metoda Metoda() pe obj1. Aici intervine polimorfismul. Deoarece Metoda() este declarată ca virtual în clasa de bază și suprascrisă în clasa derivată, se va apela versiunea metodei corespunzătoare **tipului real al obiectului**, nu tipului referinței. Tipul real al obiectului este Clasa2, deci se va apela tot metoda din Clasa2, afișând din nou **"Metoda2"**.

De ce nu se afișează Metoda1? Dacă metoda din Clasa2 ar fi fost marcată cu new (în loc de override), atunci:

- obj2.Metoda() ar fi afișat Metoda2.
- obj1.Metoda() ar fi afișat Metoda1 (comportament nepolimorfic).

Concluzia este susținută de materialul de curs "03 Programare (1).pdf", secțiunea "3.12. Polimorfism", care afirmă: "La rulare, atunci când codul client apelează metoda, CLR verifică tipul pe care obiectul îl are la rulare și invocă varianta suprascrisă a metodei virtuale." și oferă un exemplu identic în secțiunea "3.12.1. Membri virtuali".

20. Ce se va întâmpla la apelarea metodei următoare?

```
public void Metoda()  
{  
    int i=5;  
    while (i != i + 0)  
    {  
        Console.WriteLine("Hello");  
    }  
}
```

aplicația va intra în buclă infinită și va afișa la nesfârșit "Hello"
va afișa o singură dată string-ul "Hello"
va apărea o eroare de tip StackOverflow
metoda va fi părăsită fără a se afișa nimic

- Se inițializează variabila `i` cu valoarea 5.
- Se evaluează condiția buclei `while`: `i != i + 0`.
- Expresia `i + 0` este din punct de vedere matematic identică cu `i`.
- Prin urmare, condiția devine `i != i` (adică `5 != 5`), care este întotdeauna **falsă**.
- Deoarece condiția buclei `while` este falsă de la prima evaluare, corpul buclei (care conține `Console.WriteLine("Hello");`) nu se va executa niciodată.
- Programul va sări peste bucla `while` și metoda se va încheia fără a afișa nimic.

21. Ce va afișa la ieșire următoarea secvență de cod?

```
for (int i = 0; i < 5; i++)  
{  
    for (int j = 0; j < 5; j++)  
    {  
        if (i == j) break;  
        Console.Write("{0} ", j++);  
    }  
}
```

0 2 4 0 0 2 4 0 2
1 3 5 1 1 3 5 1 3
0 0 1 0 1 2 0 1 2 3

1. **Bucla exterioară (i):**
 - `i` ia valorile 0, 1, 2, 3, 4.
2. **Bucla interioară (j):**
 - Pentru fiecare `i`, `j` ia valorile 0, 1, 2, 3, 4.
 - **Condiția** `if (i == j) break;`:
 - Dacă `i` și `j` sunt egale, bucla interioară se oprește.
 - `Console.Write("{0} ", j++);`:
 - Afișează valoarea curentă a lui `j`, apoi îl incrementează cu 1.
 - `j` este **deja incrementat cu 1** în buclă (`j++`), iar la următorul pas al buclei, `j` se mai incrementează o dată cu 1 (din `for (j++)`).
3. **Analiză pentru fiecare i:**
 - `i = 0`:
 - `j = 0`: `i == j` (`0 == 0`) → `break` (nu se afișează nimic).
 - `i = 1`:
 - `j = 0`: `i != j` → afișează 0, `j` devine 1 (apoi `for` îl incrementează la 2).
 - `j = 2`: `i != j` → afișează 2, `j` devine 3 (apoi `for` îl incrementează la 4).
 - `j = 4`: `i != j` → afișează 4, `j` devine 5 (apoi `for` îl verifică pe `5 < 5` → bucla se oprește).
 - **Afișare:** 0 2 4

- $i = 2$:
 - $j = 0$: afișează 0, j devine 1 (apoi for îl incrementează la 2).
 - $j = 2$: $i == j$ ($2 == 2$) → break.
 - **Afișare: 0**
- $i = 3$:
 - $j = 0$: afișează 0, j devine 1 (apoi for îl incrementează la 2).
 - $j = 2$: afișează 2, j devine 3 (apoi for îl incrementează la 4).
 - $j = 4$: $i != j$ → afișează 4, j devine 5 (bucloa se oprește).
 - **Afișare: 0 2 4**
- $i = 4$:
 - $j = 0$: afișează 0, j devine 1 (apoi for îl incrementează la 2).
 - $j = 2$: afișează 2, j devine 3 (apoi for îl incrementează la 4).
 - $j = 4$: $i == j$ ($4 == 4$) → break.
 - **Afișare: 0 2**

4. **Rezultat final:** 0 2 4 0 0 2 4 0 2

22.Ce face metoda de mai jos?

```
public int Func(int n)
{
    if (n != 0)
    {
        return (n % 10 + Func(n / 10));
    }
    else
    {
        return 0;
    }
}
```

verifică dacă n este divizibil cu 10

returnează restul împărțirii lui n la 10

verifică dacă n este divizibil cu orice putere a lui 10

returnează suma cifrelor numărului n

Exemplu pentru $n = 1234$:

- $1234 \% 10 + \text{Func}(123) \rightarrow 4 + \text{Func}(123)$
- $123 \% 10 + \text{Func}(12) \rightarrow 3 + \text{Func}(12)$
- $12 \% 10 + \text{Func}(1) \rightarrow 2 + \text{Func}(1)$
- $1 \% 10 + \text{Func}(0) \rightarrow 1 + 0$
- **Rezultat final:** $4 + 3 + 2 + 1 = 10$

23.Ce va afișa la rulare codul de mai jos?

```
class MyApplication
{
    static void Main(string[] args)
    {
        int n = 1;
        Metoda1(n);
        Console.Write("{0}, ", n);
        Metoda2(ref n);
        Console.Write("{0}, ", n);
    }

    static void Metoda1(int n)
    {
        n += 10;
        Console.Write("{0}, ", n);
    }

    static void Metoda2(ref int n)
    {
        n = n + 10;
        Console.Write("{0}, ", n);
    }
}
```

11, 1, 21, 11,
11, 1, 11, 11,
11, 11, 21, 21,

1. `int n = 1;` în Main
 - Inițial, $n = 1$
2. `Metoda1(n)` ; (Transmitere prin valoare)
 - Se apelează `Metoda1`, trimițându-se o **copie** a valorii lui `n` (deci 1).
 - În interiorul `Metoda1`, această copie locală a lui `n` devine $1 + 10 = 11$.
 - Se afișează valoarea copiei locale: **11**, .
 - La finalul metodei, copia locală este distrusă. Variabila `n` din `Main` **rămâne neschimbată**, având tot valoarea 1.
3. `Console.Write("{0}, ", n);`
 - Se afișează valoarea curentă a variabilei `n` din `Main`, care este 1.
 - **Se afișează: 1**, .
 - Ieșirea de până acum este: 11, 1,
4. `Metoda2(ref n)` ; (Transmitere prin referință)
 - Se apelează `Metoda2`, trimițându-se o **referință** către variabila `n` din `Main`. Orice modificare în `Metoda2` va afecta direct variabila originală din `Main`.
 - În interiorul `Metoda2`, `n` (care este o referință la `n`-ul din `Main`) devine $1 + 10 = 11$.
 - Se afișează noua valoare a lui `n`: **11**, .
 - Ieșirea de până acum este: 11, 1, 11,
5. `Console.Write("{0}, ", n);`
 - Se afișează din nou valoarea variabilei `n` din `Main`. Deoarece a fost modificată prin referință în `Metoda2`, valoarea ei este acum 11.
 - **Se afișează: 11**

24.Ce va afișa la rulare programul următor?

```
public class Program
{
    public class Class1
    {
        public virtual string Metoda()
        {
            return "Class1";
        }
    }

    public class Class2 : Class1
    {
        public override string Metoda()
        {
            return "Class2";
        }
    }

    public class Class3 : Class2
    {
        public new string Metoda()
        {
            return "Class3";
        }
    }

    public static void Main()
    {
        Class1 obj = new Class3();
        Console.WriteLine(obj.Metoda());
    }
}
```

nu va rula din cauza unei erori de compilare

Class2

Class3

Class1

Ierarhia claselor și metodele:

- Class1 are o metodă virtuală Metoda() care returnează "Class1".
- Class2 moștenește Class1 și **suprascrie** (override) metoda, returnând "Class2".
- Class3 moștenește Class2 și **ascunde** (new) metoda, returnând "Class3".

- **Class1 obj = new Class3();** :: Se creează o instanță a clasei Class3, dar referința la acest obiect este stocată în Class1. Acest lucru este posibil datorită moștenirii (Class3 moștenește Class2, care la rândul ei moștenește Class1).
- **Console.WriteLine(obj.Metoda());** :: Se apelează metoda prin intermediul referinței obj de tip Class1.
 - Deoarece metoda Metoda() este declarată virtual în Class1, motorul de execuție (CLR) va căuta cea mai specifică implementare a acestei metode în ierarhia de moștenire, pornind de la tipul real al obiectului, care este Class3.
 - În Class3, metoda Metoda() este declarată cu new. Cuvântul cheie **new** întrerupe lanțul polimorfic și **ascunde implementarea moștenită**. Această metodă ar fi apelată doar dacă referința ar fi de tip Class3 (ex: Class3 obj = new Class3();). Deoarece referința noastră este de tip Class1, această metodă este ignorată.

- Motorul de execuție urcă în ierarhia de moștenire la clasa `Class2`. Aici găsește o metodă `Metoda()` declarată cu `override`. Aceasta este cea mai specifică implementare *suprascrisă* (nu ascunsă) din ierarhia obiectului.
- Prin urmare, se execută metoda din `Class2`, care returnează stringul **"Class2"**.

25.Ce va afișa la ieșire codul de mai jos?.

```
static bool Metoda1()
{
    Console.WriteLine("Metoda 1");
    return false;
}

static bool Metoda2()
{
    Console.WriteLine("Metoda 2");
    return true;
}

static void Main(string[] args)
{
    if (Metoda1() & Metoda2())
    {
        Console.WriteLine("Bloc If");
    }
}
```

Metoda 1

Bloc If

Metoda 1

Metoda 2

Bloc If

Metoda 1

Metoda 2

- Se evaluează condiția `if (Metoda1() & Metoda2())`. -se evalueaza ambele parti
- Pentru a rezolva expresia, se apelează primul operand, `Metoda1()`. Aceasta afișează la consolă **"Metoda 1"** și returnează `false`.
- Deoarece s-a folosit operatorul `&`, programul trebuie să evalueze și al doilea operand, chiar dacă primul este deja `false`.
- Se apelează `Metoda2()`. Aceasta afișează la consolă **"Metoda 2"** și returnează `true`.
- Se calculează rezultatul final al condiției: `false & true`, care este `false`.
- Deoarece condiția din `if` este `false`, blocul de cod `{ Console.WriteLine("Bloc If"); }` **nu se execută**.

26.Ce va afișa la rulare următorul program?.

```
public delegate void Del(int n);
public class TestDelegate
{
    public void VerificareParitate(int n)
    {
        if (n % 2 == 0)
            Console.WriteLine("Numar par");
        else
            Console.WriteLine("Numar impar");
    }
}
```

```

public void PatratNumar(int n)
{
    Console.WriteLine("Patrat numar: {0}", n*n);
}

class Exemplu
{
    public static void Main()
    {
        TestDelegate obj = new TestDelegate();
        Del delObj = new Del(obj.VerificareParitate);
        delObj += new Del(obj.PatratNumar);
        delObj(25);
    }
}

```

programul va rula fără probleme însă nu va afișa nimic

Numar impar

Numar impar
Patrat numar: 625

nu va afișa nimic deoarece va da eroare la rulare

Patrat numar: 625

"Un delegat este un tip care definește semnătura unei metode. [...] Delegatului îi poate fi atribuită orice metodă din orice clasă sau structură accesibilă, care corespunde semnăturii delegatului (tip returnat și parametri). Metoda poate să fie statică sau metodă a unei instanțe. [...] Delegatul trebuie să fie instanțiat cu o metodă sau expresie lambda care are tip returnat și parametri de intrare compatibili. [...] Delegații permit combinarea cu ajutorul operatorilor +, -, += și -=. Astfel, este posibil ca unei instanțe a unui delegat să îi fie atribuite mai mulți delegați (delegați multicast), în acest caz, la apelarea acestuia fiind apelați în ordine delegații atribuiți."

1. Delegatul Del:

- Este un delegat care poate referi metode cu un parametru int și return type void.

2. Crearea delegatului și adăugarea metodelor:

- Del delObj = new Del(obj.VerificareParitate); → Delegatul referă metoda VerificareParitate.
- delObj += new Del(obj.PatratNumar); → Adaugă metoda PatratNumar la lista de invocare a delegatului.

3. Apelul delegatului delObj(25):

- Se execută **toate metodele** din lista de invocare, în ordinea adăugării:
 1. VerificareParitate(25) → $25 \% 2 \neq 0$ → Afișează "Numar impar".
 2. PatratNumar(25) → Calculează $25 * 25 = 625$ → Afișează "Patrat numar: 625".

27. Ce va returna apelul `Metoda("abcd")` ?.

```
public string Metoda(string s)
{
    string rez = "";
    for (int i = s.Length - 1; i >= 0; i--)
        rez += s[i];
    return rez;
}
```

abcd
dcb
abc
abcdcba
dcba

• Inițializare:

- `string rez = ""`; (șirul rezultat este inițial gol)
- Bucula `for` pornește de la indexul final al șirului `s` (`s.Length - 1`, adică 3) și merge înapoi până la 0.

• Execuția buclei:

`i = 3` → `s[3] = 'd'` → `rez = "d"`

`i = 2` → `s[2] = 'c'` → `rez = "dc"`

`i = 1` → `s[1] = 'b'` → `rez = "dcb"`

`i = 0` → `s[0] = 'a'` → `rez = "dcba"`

28. Ce va afișa la rulare următorul program?

```
public static void Main(string[] args)
{
    int i, j = 1, k;
    for (i = 0; i < 5; i++)
    {
        k = j++ + ++j;
        Console.Write(k + " ");
    }
}
```

4 8 12 16 20
2 4 6 8 10
8 4 16 12 20
4 8 16 32 64

- Inițializare: `j = 1` (valoarea inițială).

Expresia `k = j++ + ++j`:

`j++` (post-in): Returnează valoarea curentă a lui `j` (pentru calcul), apoi îl incrementează cu 1.

`++j` (pre-increm): Incrementează `j` cu 1, apoi returnează noua valoare pentru calcul.

Ordinea operațiilor:

Se evaluează `j++` (returnează `j`, apoi `j = j + 1`).

Se evaluează `++j` (`j = j + 1`, apoi returnează `j`).

Se adună cele două valori.

- Calcul pentru fiecare iterație:

Iterația 1 (i = 0):

j = 1 (inițial).
j++ returnează 1, apoi j devine 2.
++j incrementează j la 3, apoi returnează 3.
k = 1 + 3 = 4.

Iterația 2 (i = 1):

j = 3 (de la iterația anterioară).
j++ returnează 3, apoi j devine 4.
++j incrementează j la 5, apoi returnează 5.
k = 3 + 5 = 8.

Iterația 3 (i = 2):

j = 5.
j++ returnează 5, apoi j devine 6.
++j incrementează j la 7, apoi returnează 7.
k = 5 + 7 = 12.

Iterația 4 (i = 3):

j = 7.
j++ returnează 7, apoi j devine 8.
++j incrementează j la 9, apoi returnează 9.
k = 7 + 9 = 16.

Iterația 5 (i = 4):

j = 9.
j++ returnează 9, apoi j devine 10.
++j incrementează j la 11, apoi returnează 11.
k = 9 + 11 = 20.

29.Ce va produce la ieșire următorul program?

```
static void Func1(ref int num)
{
    num = num + num;
}

static void Func2(out int num)
{
    num = num * num;
}

public static void Main(string[] args)
{
    int i = 3;
    int j = 2;
    Func1(ref i);
    Func2(out j);
    Console.WriteLine(i + " " + j);
}
```

Eroare: *Function call without creating an object*

Eroare: *Use of unassigned out parameter*

3 2

6 4

Eroarea se produce la compilare, în interiorul metodei Func2, la linia `num = num * num;`

Deoarece `num` este un **parametru out considerat nealocat** în acel punct, iar **citirea lui înainte de a-i fi atribuit o valoare este interzisă**. (compilatorul tratează un parametru out ca pe o **variabilă neinițializată** la începutul metodei, indiferent dacă variabila pasată de apelant avea sau nu o valoare.)

30. Se dă următoarea secvență de cod:

```
if (A)
    Console.WriteLine("Eroare");
else if (B)
    Console.WriteLine("Eroare");
```

Cu ce este echivalentă aceasta?

```
if (!A && A || B)
    Console.WriteLine("Eroare");

if (A || !A || B)
    Console.WriteLine("Eroare");

if (A || !A && B)
    Console.WriteLine("Eroare");
```

Mesajul "Eroare" se afișează în două cazuri:

1. Dacă A este adevărat.
2. Dacă A este fals **ȘI** B este adevărat.

31. Ce face funcția următoare?

```
public bool Metoda(string s)
{
    int min = 0;
    int max = s.Length - 1;
    while (true)
    {
        if (min > max)
        {
            return true;
        }
        if (s[min] != s[max])
        {
            return false;
        }
        min++;
        max--;
    }
}
```

verifică dacă stringul `s` conține un număr impar de caractere

verifică dacă stringul `s` este palindrom (este același indiferent de sensul în care este citit)

verifică dacă stringul `s` conține un număr par de caractere

returnează stringul `s` inversat

verifică dacă stringul `s` conține caractere duplicate

Bucula `while` compară în mod repetat caracterele de la capetele opuse ale șirului (`s[min]` și `s[max]`).

Dacă în orice moment caracterele de la capete nu sunt identice (`s[min] != s[max]`), înseamnă că șirul nu este un palindrom și metoda returnează `false`.

32. Care dintre următoarele variante este cea corectă pentru apelarea metodei `Afisare` din clasa `Test` de mai jos?

```
class Test
{
    public int Afisare(int i)
    {
        Console.WriteLine("Valoarea este " + i);
        return 0;
    }
}
```

```
delegate int del(int i);
Test t = new Test();
del = new delegate(ref Afisare);
del(10);
```

```
delegate int del(int i);
del d;
Test t = new Test();
d = new del(ref t.Afisare);
d(10);
```

```
delegate void del(int i);
Test t = new Test();
del d = new del(ref t.Afisare);
d(10);
```

33.Care dintre următoarele variante ale metodei de mai jos va determina în mod corect dacă un număr este par sau impar?

```
private bool Par(int n)
{
    return (n mod 2 == 0);
}
```

```
private bool Par(int n)
{
    return (n / 2 == 0);
}
```

```
private bool Par(int n)
{
    return (n % 2 == 0);
}
```

Verificare daca un numar este par sau impar se face prin analizarea restului impartirii la 2, care se efectueaza cu operatorului modulo (%).

34.Ce va afișa la apelare metoda de mai jos?

```
public void Metoda238()
{
    try
    {
        throw new NullReferenceException("C");
        Console.WriteLine("A");
    }
    catch (ArithmeticException e)
    {
        Console.WriteLine("B");
    }
}
```

System.NullReferenceException:C

A

System.ArithmeticException

B

System.NullReferenceException:C

System.NullReferenceException:C

B

System.NullReferenceException:C

A

B

1. **Execuția blocului try:** Programul intră în blocul try și execută prima instrucțiune: `throw new NullReferenceException("C");`. Aceasta "aruncă" o excepție de tipul `NullReferenceException`.
2. **Întreruperea execuției:** Imediat ce o excepție este aruncată, execuția normală a codului din blocul try se oprește. Prin urmare, linia `Console.WriteLine("A");` **nu** va fi niciodată executată.
3. **Căutarea unui catch compatibil:** Motorul de execuție caută un bloc catch care să poată "prinde" excepția aruncată (`NullReferenceException`).
4. **Analiza blocului catch:** Blocul catch existent este `catch (ArithmeticException e)`. Acesta este specific pentru prinderea excepțiilor de tip `ArithmeticException` sau a celor derivate din aceasta.
5. **Incompatibilitatea tipurilor:** Conform ierarhiei excepțiilor din .NET (prezentată în Figura 3.16 din "03 Programare (1).pdf"), `NullReferenceException` și `ArithmeticException` sunt tipuri diferite și nu sunt în relație de moștenire una cu cealaltă. Prin urmare, blocul catch (`ArithmeticException e`) **nu poate** prinde excepția `NullReferenceException`.
6. **Excepție neprinsă (Unhandled Exception):** Deoarece nu există niciun bloc catch compatibil în metodă, excepția rămâne "neprinsă". Acest lucru duce la terminarea programului și la afișarea detaliilor excepției neprinse în consolă. Această afișare include, de regulă, tipul excepției și mesajul asociat (în acest caz, "C").

Comportamentul este confirmat în documentul "03 Programare (1).pdf", secțiunea "3.16.4.1. Cuvântul cheie catch", care explică: "C# compară tipul obiectului excepție aruncat cu cele din clauzele catch în ordinea în care acestea apar și execută primul bloc care corespunde din punct de vedere al tipului excepției."

35. Ce efect va avea apelarea metodei următoare?

```
void Metoda(ref int a, ref int b)
{
    a = a + b;
    b = a - b;
    a = a - b;
}
```

modificarea valorilor parametrilor astfel: $a = a + b$ și $b = a - b$
valoarea parametrului `b` va rămâne neschimbată iar valoarea parametrului `a` va fi egală cu `a param b`
inversarea valorilor celor doi parametri
nici un efect

Pt ca `a` și `b` sunt transmiși prin referință (folosind cuvântul cheie `ref`), modificările făcute în interiorul metodei vor afecta direct variabilele originale din codul apelant.

36. În ce situație metoda următoare va returna `true`?

```
public bool Metoda(int n)
{
    int k = 0;
    for (int i = 1; i <= n/2; i++)
    {
        if (n % i == 0)
        {
            k++;
        }
    }
}
```

```

    if (k == 2)
        return true;
    else
        return false;
}

```

numărul n este prim

numărul n este impar

numărul n este par

numărul n este pătrat perfect

Cred ca este gresit, trebuia if (k == 1).

37.Ce va afișa la ieșire codul de mai jos?

```

static void Metoda(int val)
{
    val = 0;
}

static void Metoda1(ref int val)
{
    val = 1;
}

static void Metoda2(out int val)
{
    val = 2;
}

static void Main(string[] args)
{
    int i = 3;
    Metoda2(out i);
    Console.WriteLine(i);

    Metoda1(ref i);
    Console.WriteLine(i);

    Metoda(i);
    Console.WriteLine(i);
}

```

2
1
3

2
1
0

2
1
1

Cand apelam Metoda2, parametrul este transmis cu modificatorul out, care inseamna ca orice valoare i se atribuie in metoda se va propaga inapoi in codul apelat, adica i devine 2 dupa apelul Metoda2(out i).

Cand apelam Metoda1, parametrul e trimis cu modificatorul ref, adica prin referinta. Diferenta dintre out si ref este ca in cazul lui out, parametrul poate fi doar atribuit si nu trebuie initializat inainte de a fi trimis functiei apelate, pe cand ref trebuie initializat si poate fi atat citit cat si atribuit in functie. Dupa apelul Metoda1(ref i), variabila i va avea valoarea 1.

Cand apelam Metoda, parametrul e trimis prin valoare, adica modificarile aduse in metoda nu se vor propaga in codul care apeleaza metoda. Dupa apelul Metoda(i), variabila i va avea aceeasi valoare ca inainte de apel, adica 1.

38.Ce va calcula metoda următoare?

```
public double Metoda(int n)
{
    if (n == 1)
        return 1;
    else
        return n * Metoda(n - 1);
}
```

factorialul lui n

al n-lea număr din șirul lui Fibonacci

suma lui Gauss (1+2+...+n)

$n * (n - 1)$

$n * (n - 1) * (n - 2) * ... * 1$, care este factorialul lui n. if (n == 0) return 1; corespunde definiției $0! = 1$

39.Ce va afișa la rulare programul următor?

```
class Exemplu
{
    public int x;
    private int y;

    public void Atribuire(int a, int b)
    {
        x = a + 1;
        y = b;
    }
}

public class Program
{
    public static void Main(string[] args)
    {
        Exemplu e = new Exemplu();
        e.Atribuire(1, 1);
        Console.WriteLine(e.x + " " + e.y);
    }
}
```

2 1

programul nu va afișa nimic, deoarece va da eroare de compilare (inaccesibilitatea unui membru)

programul va compila cu succes, dar nu va afișa nimic

1 1

- În clasa Exemplu, câmpul x este declarat ca public, ceea ce înseamnă că poate fi accesat de oriunde din program.
- Câmpul y este declarat ca private. Conform materialelor de curs, un membru private are accesul limitat la tipul care îl conține. Aceasta înseamnă că y poate fi accesat doar de codul din interiorul clasei Exemplu.
- În metoda Main, care se află în afara clasei Exemplu, linia Console.WriteLine(e.x + " " + e.y); încearcă să acceseze atât e.x, cât și e.y.
- Accesarea e.x este validă, deoarece x este public.
- Accesarea e.y este **invalidă**, deoarece metoda Main se află în afara clasei Exemplu și nu are permisiunea de a accesa membrii ei privați.

40.Ce va fi afișat în consolă la rularea următorului program?

```
class Class1
{
    public Class1()
    {
        Console.WriteLine("constructor Class1");
    }

    public void afisare()
    {
        Console.WriteLine("1");
    }
}

class Class2 : Class1
{
    public Class2()
    {
        Console.WriteLine("constructor Class2");
    }

    public new void afisare()
    {
        Console.WriteLine("2");
    }
}

class Program
{
    static void Main(string[] args)
    {
        Class2 b = new Class2();
        Class1 a = b;
        a.afisare();
    }
}
```

constructor Class2

constructor Class1

1

constructor Class1

constructor Class2

1

constructor Class1

constructor Class2

2

Class2 b = new Class2(); :: Se creează o instanță a clasei derivate Class2.

- **Regula de moștenire a constructorilor:** Atunci când se creează un obiect al unei clase derivate, constructorul clasei de bază (Class1) este apelat **întotdeauna primul**. Această regulă este menționată în documentul "03 Programare (1).pdf".
- Se execută constructorul din Class1, afișând "**constructor Class1**".
- Apoi, se execută constructorul din Class2, afișând "**constructor Class2**".

Class1 a = b; :: Se creează o referință a de tipul clasei de bază Class1, care indică spre obiectul de tip Class2 creat anterior.

a.afisare(); :: Se apelează metoda afisare() prin intermediul referinței a.

- **Ascunderea metodei (new):** În clasa `Class2`, metoda `afisare()` este declarată cu cuvântul cheie `new`, nu `override`. Acest lucru înseamnă că metoda din `Class2` o **ascunde** pe cea din `Class1`, dar nu o suprascrie (nu participă la polimorfism).
- **Rezoluția apelului:** Deoarece metoda nu este virtuală/suprascrisă, compilatorul decide ce metodă să apeleze pe baza **tipului referinței**, nu a tipului real al obiectului. Deoarece referința este de tip `Class1`, se va apela metoda `afisare()` din clasa `Class1`.
- Se afișează **"1"**.

41. Se dă următoarea clasă:

```
class Clasa1
{
    public Clasa1() { }

    public Clasa1(int valoare) { }
}
```

Cum ar putea fi modificat primul constructor astfel încât să îl apeleze pe cel de-al doilea (cu parametru)?

```
public Clasa1()
{
    this(10);
}

public Clasa1()
{
    return new this(10);
}

public Clasa1 : this(10)
{
}
```

Curs, pagina 75: "Un constructor poate invoca un alt constructor al aceluiași obiect prin utilizarea cuvântului cheie `this`. La fel ca și `base`, `this` poate fi utilizat cu sau fără parametri, oricare din parametrii constructorului fiind disponibili acestuia."

42. Ce va afișa la rulare programul următor?

```
class Program
{
    class Baza
    {
        public void Afisare()
        {
            Console.WriteLine("Afisare Baza" + ", ");
        }
    }

    class Deriv1 : Baza
    {
        private void Afisare()
        {
            Console.WriteLine("Afisare Deriv1" + ", ");
        }
    }

    class Deriv2 : Deriv1
    {
        new void Afisare()
        {
            Console.WriteLine("Afisare Deriv2" + ", ");
        }
    }
}
```



```
static void Main(string[] args)
{
    Deriv2 d = new Deriv2();
    d.Afisare();
}
}
```

Afisare Deriv2,

Afisare Baza,

Afisare Deriv2, Afisare Deriv1, Afisare Baza,

În mod normal, atributul new pentru metoda din clasa derivată Deriv2 ar face să fie apelată acea metodă dacă obiectul este de tip Deriv2, dar neavând identificatorul de acces precizat, e luat implicit ca private. Din cauza asta, este apelată metoda clasei Baza care are identificatorul public.

43. Clasa `Deriv` moștenește clasa `Baza`. Clasa `Baza` are un constructor cu doi parametri. Cum trebuie declarat un constructor din clasa `Deriv`?

```
public Deriv(int val, int val2) : base(val1, val2)
{
}
public Deriv(int val, int val2) : MyBase(val1, val2)
{
}
public Deriv(int val, int val2) : Baza(val1, val2)
{
}
```

În C#, un constructor dintr-o clasă derivată apelează un constructor specific din clasa de bază folosind cuvântul cheie `base`.

44. Se consideră două clase: `Cls1`, `Cls2` și două interfețe: `Interf1`, `Interf2`. Care din următoarele declarații ale clasei `Cls3` va genera eroare la compilare?

```
class Cls3 : Interf1, Interf2
{
}
class Cls3 : Cls1, Cls2
{
}
class Cls3 : Cls1, Interf1
{
}
```

- **Moștenirea de la clase:** O clasă în C# poate moșteni de la **o singură clasă de bază**. Încercarea de a moșteni de la mai multe clase (`Cls1` și `Cls2` în acest caz) se numește moștenire multiplă și nu este permisă în C#.

- **Implementarea interfețelor:** O clasă poate implementa **una sau mai multe interfețe**. Documentul menționează: "Tipurile `class` și `struct` pot implementa interfețe multiple." Prin urmare, declarația

`class Cls3 : Interf1, Interf2` este validă.

- **Moștenire și implementare combinate:** O clasă poate moșteni o singură clasă de bază și, în același timp, poate implementa mai multe interfețe. Condiția este ca numele clasei de bază să apară primul în listă. Așadar, declarația

`class Cls3 : Cls1, Interf1` este validă.

45.Care dintre următoarele secvențe de cod va genera eroare la compilare?

```
public class Cls1
{
    protected static double val = 3.14;
    public void Print()
    {
        Console.WriteLine(val);
    }
}

public class Cls2
{
    private int val = 0;
    public static void Print()
    {
        Console.WriteLine(val);
    }
}

public static class Cls3
{
    private const int val = 0;
    public static void Print()
    {
        Console.WriteLine(val);
    }
}
```

public class Cls1: Acest cod este **valid**. O metodă de instanță (`Print`) poate accesa fără probleme un membru static (`val`) al aceleiași clase.

public class Cls2: Acest cod va genera o **eroare de compilare**.

- Câmpul `val` este un membru de instanță (non-static), ceea ce înseamnă că aparține unui obiect specific creat din clasa `Cls2`.
- Metoda `Print` este o metodă statică. Metodele statice aparțin clasei în sine, nu unei instanțe anume, și pot fi apelate fără a crea un obiect (`Cls2.Print()`).
- Regula fundamentală, menționată și în documentul "03 Programare (1).pdf", este că o metodă statică nu poate accesa direct un membru de instanță (non-static), deoarece nu știe de la ce instanță a obiectului să ia valoarea. Textul specifică: "Metodele și proprietățile statice nu pot accesa câmpurile și evenimentele non-statice din aceeași clasă".

public static class Cls3: Acest cod este **valid**.

- Într-o clasă statică, toți membrii trebuie să fie statici.
- Un câmp declarat `const` (constantă) este implicit static.
- Prin urmare, metoda statică `Print` poate accesa fără probleme câmpul constant (implicit static) `val`.

46.Secvența de cod de mai jos:

```
private void button1_Click(object sender, EventArgs e)
{
    int x = Convert.ToInt32(textBox.Text);
    if (x <= 0)
    {
        string n = "negativ sau zero";
        textBox.Text = n;
    }
    else
        for(int n = 1; n <= x; n++)
            listBox.Items.Add(n);
}
```

va functiona fara erori

va genera avertisment la compilare din cauza utilizarii in cadrul metodei a doua variabile cu acelasi nume

va genera eroare la compilare din cauza utilizarii in cadrul metodei a doua variabile cu acelasi nume

Cheia pentru a răspunde la această întrebare este înțelegerea conceptului de **domeniu de vizibilitate (scope)** al variabilelor în C#.

1. **Prima variabilă n:** `string n = "negativ sau zero";` este declarată în interiorul blocului `if`. Conform regulilor limbajului, domeniul ei de vizibilitate este limitat strict la acest bloc (între acoladele `{}`). Ea nu există în afara acestui bloc.
2. **A doua variabilă n:** `for(int n = 1; ...)` este declarată în antetul buclei `for`. Domeniul ei de vizibilitate este limitat la bucla `for` (incluzând corpul și antetul acesteia).

Deoarece cele două variabile `n` sunt declarate în domenii de vizibilitate diferite și care nu se suprapun (una există doar în blocul `if`, cealaltă doar în bucla `for` din blocul `else`), nu există niciun conflict de nume. Compilatorul le tratează ca pe două variabile distincte, complet separate.

47.Ce se va afisa la rulare secventa de code de mai jos?

```
int[] arr1 = { 0, 1, 2, 3 };
int[] arr2 = arr1;
arr2[0] = 7;

foreach( int elem in arr1)
    Console.Write("{0} ", elem);
Console.WriteLine();

foreach(int elem in arr2)
    Console.Write("{0} ", elem);
```

0 1 2 3

7 1 2 3

7 1 2 3

7 1 2 3

7 1 2 3

0 1 2 3

Cheia pentru a înțelege acest rezultat este faptul că vectorii (arrays) în C# sunt **tipuri referință**.

1. `int[] arr1 = { 0, 1, 2, 3 };:` Se creează un vector în memorie și variabila `arr1` reține o referință (o adresă) către acest vector.
2. `int[] arr2 = arr1;:` Această linie **nu** creează un nou vector. În schimb, copiază referința din `arr1` în `arr2`. Acum, ambele variabile, `arr1` și `arr2`, indică spre **același vector** din memorie. Acest comportament este explicat în documentul "03 Programare (1).pdf" în secțiunea "3.4.7. Copierea vectorilor", care specifică: "...un vector .NET este un tip referință.".
3. `arr2[0] = 7;:` Se modifică primul element al vectorului accesat prin referința `arr2`. Deoarece `arr1` indică spre același vector, modificarea este vizibilă și prin `arr1`. Vectorul din memorie devine `{ 7, 1, 2, 3 }`.
4. **Prima buclă foreach:** Parcurge și afișează elementele din `arr1`, care este acum `{ 7, 1, 2, 3 }`. Se afișează `7 1 2 3`.
5. **A doua buclă foreach:** Parcurge și afișează elementele din `arr2`, care este același vector. Se afișează din nou `7 1 2 3`.

48.Ce se va afisa in consola dupa executarea urmatoarei secvente de cod?

```
int a = 0, b = 0, c = 0;
try
{
    a = b /c;
}
catch(Exception ex)
{
    Console.WriteLine(ex.Message);
    try
    {
        b = a /c;
    }
    catch
    {
        return;
    }
}
finally
{
    Console.WriteLine("Finally.");
}
```

Attempted to divide by zero.

Finally.

Attempted to divide by zero.

Attempted to divide by zero.

Attempted to divide by zero.

Execuția programului se desfășoară în următorii pași:

1. **Blocul try exterior:** Se încearcă executarea operației $a = b / c$; (adică $0 / 0$). Aceasta aruncă o excepție de tipul `DivideByZeroException`.
2. **Blocul catch exterior:** Excepția este prinsă de blocul `catch(Exception ex)`. Se execută prima instrucțiune din acest bloc: `Console.WriteLine(ex.Message);`. Mesajul standard pentru această excepție este "Attempted to divide by zero."
3. **Blocul try interior:** Execuția continuă în interiorul blocului `catch` și se intră în blocul `try` interior. Se încearcă din nou o împărțire la zero: $b = a / c$; (adică $0 / 0$), care aruncă o nouă excepție `DivideByZeroException`.
4. **Blocul catch interior:** Această a doua excepție este prinsă de blocul `catch` interior. Acesta conține o singură instrucțiune: `return;`. Instrucțiunea `return` ar trebui să oprească execuția metodei.
5. **Blocul finally:** Aici intervine regula cheie. Blocul `finally` se execută **întotdeauna** înainte ca metoda să fie părăsită, indiferent dacă acest lucru se întâmplă normal, printr-o excepție neprinsă sau printr-o instrucțiune `return` dintr-un bloc `try` sau `catch`.
 - o Comportamentul este confirmat în documentul "03 Programare (1).pdf", secțiunea "3.16.4.2. Cuvântul cheie `finally`" : "Blocul `finally` rulează ... chiar dacă metoda este părăsită ca urmare a apelului unei comenzi `return`."
 - o Prin urmare, înainte ca metoda să se oprească din cauza instrucțiunii `return`, se execută codul din `finally`: `Console.WriteLine("Finally.");`.

După executarea blocului `finally`, metoda se încheie conform instrucțiunii `return`.

49.Urmatoarea secventa de cod va afisa:

```
class Program
{
    static void Main(string[] args)
    {
        int i = 0;
        int[] v = { 1, 2, 3, 4, 5, 6, 7, 8 };
        try
        {
            for (; i < v.Length; i++)
                Console.Write(v[i]);
        }
        catch
        {
            Console.WriteLine("Mesaj eroare!");
        }
    }
}
```

1234567

Mesaj eroare

12345678

Eroare

Eroare, instructiunea `for` nu este scrisa corect

1. **Inițializarea:** Se declară un vector `v` cu 8 elemente (de la index 0 la 7) și o variabilă `i` inițializată cu 0.
2. **Sintaxa `for`:** Bucula `for (; i < v.Length; i++)` este scrisă corect. Faptul că secțiunea de inițializare este goală este permis în C#, deoarece variabila `i` a fost deja inițializată înaintea buclei.
3. **Execuția buclei:** Bucula va itera cât timp `i` este mai mic decât lungimea vectorului (`v.Length`), care este 8. Astfel, `i` va lua pe rând valorile 0, 1, 2, 3, 4, 5, 6 și 7.
4. **Afișarea:** În interiorul buclei, `Console.Write(v[i])` va afișa elementul de la indexul curent `i`. Deoarece se folosește `Console.Write`, caracterele vor fi afișate unul după altul, fără spații sau trecere la rând nou.
5. **Blocul `try-catch`:** Pe parcursul execuției buclei, indexul `i` nu depășește niciodată limitele vectorului (ultima valoare validă este 7, pentru `v[7]`). Prin urmare, nu se va produce nicio eroare (`IndexOutOfRangeException` sau alta), iar blocul `catch` nu va fi executat niciodată.

50.Urmatoarea secventa de cod are ca rezultat:

```
class Program
{
    public static int validare(string m)
    {
        for (int i = 0; i < m.Length; i++)
            if (m[i] >= 'a' && m[i] <= 'z')
                return 0;
        return 1;
    }

    static void Main(string[] args)
    {
        string[] rezultat = { "Da", "Nu" };
        string mesaj = "Vacanta";
        Console.WriteLine(rezultat[validare(mesaj)]);
    }
}
```

eroare

Da

eroare, variabila rezultat nu este initializata corect
nu

51.Urmatoarea secventa de cod va afisa:

```
class Program
{
    abstract class A
    {
        public abstract int metoda1();
    }

    class B : A
    {
        public override int metoda1()
        {
            return 72;
        }
    }

    static void Main(string[] args)
    {
        B a = new B();
        A l = a;
        Console.WriteLine("{0}", l.metoda1());
    }
}
```

eroare, lipseste implementarea metoda1() in clasa A

nimic, lipseste implementarea metoda1() in clasa A

nimic, lipseste abstract inainte de class B

eroare

72

Deși atribuirea `l = a` face un cast de la `B` la `A`, `A` este abstractă și metoda `metoda1` este suprascrisă de implementarea din `B`, care va fi executată.

1. **Clasa Abstractă A:** Clasa `A` este declarată `abstract` și conține o metodă abstractă `metoda1()`. O metodă abstractă nu are un corp de implementare în clasa abstractă; ea doar definește o semnătură pe care clasele derivate non-abstracte sunt obligate să o implementeze.
2. **Clasa Derivată B:** Clasa `B` moștenește clasa `A` și **suprascrie** (`override`) metoda `metoda1()`, oferind o implementare concretă care returnează valoarea `72`. Aceasta este implementarea corectă și obligatorie pentru o clasă concretă ce derivă dintr-una abstractă.
3. **Execuția în Main:**
 - o `B a = new B();` Se creează o instanță validă a clasei `B`.
 - o `A l = a;` Se creează o referință `l` de tipul clasei de bază abstracte `A`, care indică spre obiectul de tip `B`. Acest lucru este permis de polimorfism.
 - o `Console.WriteLine("{0}", l.metoda1());` Se apelează metoda `metoda1()` prin referința la clasa de bază. Datorită polimorfismului (metoda fiind abstractă în bază și suprascrisă în derivată), se va executa implementarea din clasa obiectului real, adică din clasa `B`.

52.Urmatoarea secventa de cod are ca rezultat:

```
private void form1_Load(object sender, EventArgs e)
{
    for (int i = 50; i < 350; i = i + 50)
    {
        for (int j = 50; j < 350; j = j + 50)
        {
            if (i == j)
            {
                Label l = new Label();
                Controls.Add(l);
                l.Location = new Point(j + j, i);
                l.Text = "1";
                l.ForeColor = Color.FromName("red");
            }
            else
            {
                Button b = new Button();
                Controls.Add(b);
                b.Location = new Point(j + j, i);
                b.Text = "0";
                b.ForeColor = Color.FromName("blue");
            }
        }
    }
}
```

Eroare

generarea dinamica a unei matrice patratice 7x7, care are pe diagonala principala Textbox-uri cu textul 1 de culoare rosie, iar pe restul elementelor butoane cu text de culoare albastra

generarea dinamica a unei matrice patratice 6x6, care are pe diagonala secundara label-uri cu textul 1 de culoare rosie, iar pe restul elementelor butoane cu text de culoare albastra

generarea dinamica a unei matrice patratice 6x6, care are pe diagonata secundara si sub aceasta label-uri cu textul 1 de culoare rosie iar pe restul elementelor butoane cu text

Generarea dinamica a unei matrice patratice 6x6, care are pe diagonala principala label-uri cu textul 1 de culoare rosie, iar pe restul elementelor butoane cu text de culoare albastra

53. Ce va afisa la rulare programul urmator?

```
class Program
{
    class Figure
    {
        public void Afisare()
        {
            Console.Write("Afisare Figura, ");
        }
    }

    class Poligon : Figure
    {
        private void Afisare()
        {
            Console.Write("Afisare Poligon, ");
        }
    }
}
```

```

class Triunghi : Poligon
{
    private void Afisare()
    {
        Console.Write("Afisare Triunghi, ");
    }
}

static void Main(string[] args)
{
    Poligon p = new Poligon();
    Triunghi t = new Triunghi();
    p.Afisare();
    t.Afisare();
}
}

```

Afisare Figura, Afisare Figura,
 Afisare Poligon, Afisare Poligon,
 Afisare Poligon, Afisare Triunghi,
 nu va afisa nimic deoarece va da eroare la compilare

1. Analiza clasei Poligon:

Clasa Poligon moștenește metoda publică Afisare() de la clasa Figure. În același timp, Poligon definește propria sa metodă private void Afisare(). Deoarece este private, aceasta este accesibilă doar din interiorul clasei Poligon.

2. Analiza clasei Triunghi:

Clasa Triunghi moștenește de la Poligon. La rândul ei, definește o metodă private void Afisare(), care este accesibilă doar din interiorul clasei Triunghi.

3. Analiza metodei Main:

- **p.Afisare();** Variabila p este de tip Poligon. Când se încearcă apelarea metodei Afisare() din exterior (din Main), compilatorul găsește metoda Afisare() definită în clasa Poligon. Deoarece aceasta este private, compilatorul va genera o eroare de compilare, semnalând că metoda este inaccesibilă din cauza nivelului său de protecție.
- **t.Afisare();** Similar, variabila t este de tip Triunghi, iar metoda Afisare() definită în această clasă este private. Apelul din Main va genera, de asemenea, o eroare de compilare.

Regula de bază, confirmată și în documentul "03 Programare (1).pdf", este că membrii declarați private au "accesul limitat la tipul care conține membrul". Prin urmare, ambele apeluri din Main sunt invalide, iar programul nu va compila.

54.Ce va afisa la rulare programul urmator?

```

class Exemplu
{
    public int x;
    private int y;
    protected void Atribuire(int a, int b)
    {
        x = a + 1;
        y = b;
    }
}

```



```

public class Program
{
    public static void Main(string[] args)
    {
        Exemplu e = new Exemplu();
        e.Atribuire(-1, 1);
        Console.WriteLine(e.x + " " + e.y);
    }
}

```

programul nu va afisa nimic deoarece va da eroare la compilare (inaccesibilitatea unui membru)

-1 2

0 1

programul se va compila cu succes, dar nu va afisa nimic

1. **Accesarea metodei protected:** Metoda Atribuire este declarată protected în clasa Exemplu.

Conform materialelor de curs, un membru

protected este accesibil doar în interiorul clasei care îl definește și în clasele care derivă din aceasta.

Deoarece clasa

Program (unde se află metoda Main) nu moștenește clasa Exemplu, apelul e.Atribuire(-1, 1); este invalid și va cauza o eroare de compilare.

2. **Accesarea câmpului private:** Câmpul y este declarat private în clasa Exemplu. Un membru

private poate fi accesat doar din interiorul clasei în care este declarat. Încercarea de a citi valoarea

e.y din metoda Main este, de asemenea, invalidă și va contribui la eroarea de compilare.

55.Ce se va intampla la rularea aplicatiei urmatoare?

```

public static void Metoda(int n)
{
    int i = n++;
    while (i == n)
    {
        Console.WriteLine("Hello");
    }
}

public static void Main(string[] args)
{
    Metoda(5);
}

```

va aparea o eroarea de tip *StackOverflow*

va afisa de 5 ori stringul "Hello"

metoda va fi parasita fara a se afisa nimic

aplicatia va intra in bucla infinita si va afisa la nesfarsit stringul "Hello"

va afisa o singura data stringul "Hello"

Rezultatul rularii este: ". Variabilei i ii e atribuita valoarea lui n, dar apoi n este incrementat, deci i == n va fi totdeauna falsa. Prin urmare, nu se va afisa nimic.

56.Care dintre urmatoarele variante ale metodei de mai jos va determina in mod corect daca un numar este impar?

```
private bool Impar(int n)
{
    return (n / 2 != 0);
}
private bool Impar(int n)
{
    return (Math.Abs(n) != 0);
}
private bool Impar(int n)
{
    return (n % 2 != 0);
}
private bool Impar(int n)
{
    return (n mod 2 != 0);
}
```

57.Ce efect va avea urmatoarea secventa de cod?

```
private void pictureBox1_Paint(object sender, PaintEventArgs e)
{
    e.Graphics.DrawLine(new Pen(Color.Black, 5f), 10, 10, 50, 50);
}
```

desenarea unei linii care nu va fi vizibila

desenarea unei linii de grosime 5 pixeli

desenarea unei linii foarte fina de grosime mai mica de 1 pixel, approximate prin *antialiasing*

Comanda va desena o linie (DrawLine), cu un pen de culoare neagra si grosime 5 pixeli (Pen(Color.Black, 5f)).

58.Pentru a desena o linie in interiorul unui control PictureBox vom scrie urmatorul cod:

```
Graphics gr;
gr = pictureBox.CreateGraphics();
gr.DrawLine(Pens.Black, 10, 10, 100, 100);

Graphics gr = new Graphics();
pictureBox.CreateGraphics(gr);
gr.DrawLine(Pens.Black, 10, 10, 100, 100);

Graphics gr = new Graphics();
gr = new pictureBox.CreateGraphics();
gr.DrawLine(Pens.Black, 10, 10, 100, 100);
```

Pentru a desena o linie în interiorul unui PictureBox în C#, se folosește metoda CreateGraphics() care returnează un obiect de tip Graphics legat de acel control. Apoi, metoda DrawLine() este apelată pe acest obiect.

59.Ce va afisa secventa de cod de mai jos?

```
int val;
Console.WriteLine("Valoarea este: {0}", val);
Valoarea este: 0
Valoarea este: null
```

nu va afisa nimic deoarece va genera eroare la compilare din cauza utilizarii unei variabile neinitializate

1. int val; - Variabila locală val este declarată, dar nu i se atribuie nicio valoare.
2. Console.WriteLine("Valoarea este: {0}", val); - Această linie încearcă să citească valoarea variabilei val pentru a o afișa.

Deoarece se încearcă utilizarea variabilei `val` înainte de a fi inițializată, compilatorul C# va genera o eroare, iar programul nu va putea fi compilat sau rulat.

Acest comportament este confirmat în documentul "03 Programare (1).pdf", secțiunea "3.2.1.3. Inițializarea variabilelor", care afirmă:

"Dacă încercați să folosiți în program o variabilă înainte de a-i fi atribuit o valoare, compilatorul C# va genera eroare."

60.Ce se va afișa la rulare următoarea secvență de cod?

```
int i;  
for (i = 0; i <= 5; i++);  
Console.WriteLine(i);
```

6
5
4

Deoarece apare `;` chiar după instrucțiunea `for`, aceasta doar va incrementa contorul `i` care, fiind declarat în afara `for`, va persista după terminarea buclei. Dacă era `int` în `for` ar fi dat eroare.

61.Secvența de cod de mai jos va afișa următorul rezultat:

```
int a = 10;  
string b = "20";  
float c = 30f;  
Console.WriteLine("{1}, {0}, {2}", a, b, c);
```

20, 10, 30
10, 20, 30
10, "20", 30

Instrucțiunea de afișare folosește un string formatat cu indici numerici, care indică pozițiile parametrilor furnizați după șablon. Astfel, se vor insera în ordine variabilele cu indicii 1, 0 și 2, adică 20, 10, 30.

62.Se considera următoarea declarativă: `int[] sir = new int[10];`

sirul va conține 10 elemente, primul cu indexul 1 iar ultimul cu indexul 10
sirul va conține 10 elemente, primul cu indexul 0 iar ultimul cu indexul 9
sirul va conține 11 elemente, primul cu indexul 0 iar ultimul cu indexul 10

63.Cum se poate face declararea, instanțierea și inițializarea unui vector cu 4 elemente pe o singură linie de cod?

```
int vector[] = new int[4] { 1, 2, 3, 4 };  
int[] vector = new int[1, 2, 3, 4];  
int[] vector = new int[] { 4, 5, 6, 7 };
```

`int vector[]` e gresit pentru că parantezele de vector trebuie să fie în dreptul tipului de dată pe care îl va avea vectorul. `new int[1, 2, 3, 4]` e gresit pentru că între parantezele drepte ar trebui să fie dimensiunea vectorului la instanțiere, nu lista de valori.

64.Ce va returna apelul `Metoda("zyx");`?

```
public string Metoda(string s)  
{  
    string rez = "";  
    for (int i = s.Length - 1; i >= 0; i--)  
        rez += s[i];  
    return rez;  
}
```

xy
zy
zyx
xyz
zyxyz

65. In urmatoarea secventa de cod se doreste declararea unui obiect de tip `Inginer` in clasa `Student`:

```
namespace Persoane
{
    Class Student
    { }

    namespace Angajati
    {
        Class Inginer
        { }
    }
}
```

acest lucru nu este posibil deoarece clasa `Student` nu are acces la clasa `Inginer`

acest lucru este posibil prin declaratia `Angajati.Inginer obj`

acest lucru este posibil prin declaratia `Inginer obj`

În C#, namespace-urile pot fi imbricate. Însă, deoarece te afli deja în `Persoane`, este suficient `Angajati.Inginer obj = new Angajati.Inginer();`.

66. In urma executarii secventei de cod de mai jos care dintre afirmatii este adevarata?

```
Private void button1_Click(object sender, EventArgs e)
{
    Form2 f = new Form2();
    f.Opacity = 1;
    f.Show();
}
```

fereastra va fi afisata, insa va fi complet invizibila utilizatorului

fereastra va fi afisata cu opacitate de 1%

fereastra va fi afisata cu opacitate 100%

Curs, pagina 130: "Proprietatea `Opacity` determină opacitatea unei ferestre. În modul design, fereastra de proprietăți arată valoarea acestei proprietăți sub formă de procentaj, unde 100% semnifică faptul că formularul este complet opac, iar 0% că fereastra este complet transparentă. În timpul rulării trebuie să tratați valoarea proprietății `Opacity` ca număr în virgulă mobilă cu valori între 0.0 (complet transparent) și 1.0 (complet opac)."

Gradul de transparență (0% - 100%) a unei ferestre se poate indica prin setarea proprietății:

`Transparency`
`TransparencyKey`
`Opacity`

Curs, pagina 130: "Proprietatea `Opacity` determină opacitatea unei ferestre. În modul design, fereastra de proprietăți arată valoarea acestei proprietăți sub formă de procentaj, unde 100% semnifică faptul că formularul este complet opac, iar 0% că fereastra este complet transparentă."

Pentru ce este folosit în C# cuvântul cheie `checked`?

verificarea faptului că variabilele implicate în operații aritmetice sunt inițializate înainte de utilizare
verificarea compatibilității conversiilor între diferite tipuri de date

verificarea depășirilor (overflow) la efectuarea conversiilor și operațiilor aritmetice

Curs, pagina 45: "C# pune la dispoziția programatorilor cuvântul cheie `checked`. Dacă veți cuprinde una sau mai multe instrucțiuni într-un bloc `checked`, atunci compilatorul C# va verifica situațiile în care pot apărea depășiri la operații de adunare, scădere, înmulțire sau împărțire a două valori numerice. Astfel, la apariția unei astfel de depășiri veți primi o excepție de tipul `System.OverflowException` cu mesajul: *'Arithmetic operation resulted in an overflow'.*"

Care este specificatorul de acces implicit pentru clase?

`internal`
`private`
`public`
`protected`

Curs, pagina 71: "Modificatorii de acces sunt opționali atât pentru membri cât și pentru clase, **modificatorul implicit** fiind cel mai restrictiv disponibil, în acest caz, ***private***."

Care este specificatorul de acces care permite doar claselor derivate să acceseze membrii din clasa de bază?

`protected`
`private`
`public`
`internal`

protected: accesul este limitat la clasa care conține membrul sau la tipurile derivate din aceasta.

private: accesul este limitat la tipul care conține membrul.

public: accesul nu este restricționat.

internal: accesul este limitat la ansamblul curent

Care este specificatorul de acces care îi permite unei clase să își ascundă membrii față de alte clase, cu excepția claselor derivate din cadrul aceleiași aplicații?

`internal`
`private`
`public`
`protected`
`protected internal`

Conform materialelor de curs, specificatorul de acces care limitează accesul la clasa care conține membrul și la clasele derivate din aceasta, **dar numai în cadrul aceleiași aplicații (ansamblu)**, este ***private protected***.

În documentul "03 Programare (1).pdf", la pagina 39, sunt definite nivelurile de accesibilitate:

- **protected**: "accesul este limitat la clasa care conține membrul sau la tipurile derivate din aceasta." (Această opțiune permite accesul și claselor derivate din alte ansambluri/aplicații).
- **internal**: "accesul este limitat la ansamblul curent." (Această opțiune permite accesul *tuturor* claselor din același ansamblu, nu doar celor derivate).
- **private protected**: "accesul este limitat la tipul care conține membrul sau la tipurile din ansamblul curent derivate din clasa care conține membrul (numai din ansamblul curent)."

Modificatorul de acces `internal`:

limiteaza accesul numai la clasa curenta

limiteaza accesul numai la ansamblul curent

limiteaza accesul numai la fisierul sursa curent

Curs, pagina 39: "internal: accesul este limitat la ansamblul curent."

Care dintre următoarele afirmații este falsă?

metodele statice pot accesa numai membri statici

metodele de instanță nu pot accesa membri statici

metodele statice nu pot accesa membri de instanță

O metodă de instanță (non-statică) poate accesa membrii statici ai clasei sale.

"Metodele și proprietățile statice nu pot accesa câmpurile și evenimentele non-statice din aceeași clasă..."

Metodele statice nu pot accesa membri de instanță

Care dintre următoarele afirmații este adevărată?

metodele de extensie se declară prin adăugarea în definiția lor a cuvântului cheie `extend`

metodele de extensie trebuie să returneze obiectul `this`

metodele de extensie pot fi definite numai ca membri ai claselor statice

Curs, pagina 67: "Puteți defini metode de extensie numai ca membri ai claselor statice."

O interfață poate declara:

variabile, metode și proprietăți

variabile, metode, evenimente și delegați

metode, proprietăți și evenimente

Curs, pagina 78: "Astfel, o interfață poate declara următoarele categorii de membri: constante, metode, proprietăți, operatori, indexatori, evenimente, constructori statici, respectiv alte tipuri încuibate."

Care dintre afirmațiile următoare este corectă în legătură cu interfețele din C#?

interfețele sunt declarate folosind cuvântul cheie `interface`

metodele interfețelor sunt publice

toate afirmații sunt corecte

niciuna dintre afirmații este corectă.

Curs, pagina 79: "Pentru membrii interfețelor nu pot fi specificați modificatori de acces (vor fi implicit publici), deoarece accesibilitatea este controlată la nivelul interfeței (la fel ca și clasele, interfețele sunt publice sau interne, însă în cazul în care sunt încuibate pot avea orice modificador de acces). La implementare membrii interfețelor vor fi considerați, în mod implicit, tot publici."

O interfata se declara folosind cuvantul-cheie `interface`, la fel cum clasele folosesc `class`.

Care din urmatoarele afirmatii este falsa?

o interfata poate specifica mai multe interfete de baza

o clasa poate implementa direct numai o singura interfata

o clasa poate mosteni direct numai o singura clasa de baza

Curs, pagina 82: "Interfețele suportă moștenirea, însă aceasta nu este la fel ca moștenirea claselor. Sintaxa este similară, însă o interfață poate specifica mai multe interfețe de bază, deoarece C# suportă moștenire multiplă pentru interfețe."

Putem declara metode `protected` într-o interfață?

nu

da, dar numai dacă există tipuri de dată care implementează acea interfață
da, indiferent de situație

Curs, pagina 79: "Pentru membrii interfețelor nu pot fi specificați modificatori de acces (vor fi implicit publici), deoarece accesibilitatea este controlată la nivelul interfeței (la fel ca și clasele, interfețele sunt publice sau interne, însă în cazul în care sunt încuibate pot avea orice modificador de acces). La implementare membrii interfețelor vor fi considerați, în mod implicit, tot publici."

Proprietatile unui obiect pot fi:

proprietati numerice, proprietati text si proprietati booleene

proprietati simple, proprietati locale si proprietati `system`

proprietati compuse, proprietati restrictionate si proprietati de colectie

În .NET (și în general în programarea orientată pe obiecte), proprietățile unui obiect pot fi clasificate după structura și comportamentul lor:

- proprietăți compuse (complexe): sunt proprietăți care conțin alte obiecte;
- proprietăți restricționate (restricted): au acces limitat (ex: *read-only*, *write-only*, *private set*);
- proprietăți de colecție (collection): permit accesul la un grup de obiecte.

Metodele si proprietatile statice nu pot accesa:

membrii statici din clasa de baza

campuri non-statice din aceeași clasa

nicio instanță a vreunei clase

Curs, pagina 72: "Metodele și proprietățile statice nu pot accesa câmpurile și evenimentele non-statice din aceeași clasă și nici nu pot accesa vreo instanță a unei clase decât dacă aceasta este transmisă explicit între parametrii metodei."

Care dintre afirmațiile următoare este falsă?

într-o metodă statică nu putem folosi cuvântul `this`

metodele de extensie sunt metode statice

în declarația unui indexator nu putem folosi cuvântul `this`

Declarația unui indexator este de forma: `public float this[int index] {}`

Metodele de **extensie sunt metode statice**, care primesc ca prim parametru o referință la obiectul pe care îl extind:

```
public static int WordCount(this string s) {}
```

În **metodele statice nu se poate folosi `this`**, pentru ca metodele statice aparțin clasei, nu unui obiect.

Care dintre următoarele afirmații este adevărată?

clasa care declară un indexator trebuie să declare cel puțin un constructor static

în cazul indexatorilor, C# impune limitarea ca tipul indexului să fie `int`

în C# puteți declara indexatori multidimensionali

Curs, pagina 70: "C# nu limitează tipul indexului la `int`." Clasele care utilizează indexatorii nu trebuie să aibă constructor static.

Membrii claselor, incluzand aici si clasele incuivate, pot avea urmatorii modificatori de acces:

public, protected, internal, private internal **sau** private

public, protected internal, protected, internal **sau** private

public, public internal, protected, internal **sau** private

Curs, pagina 71: "Membrii claselor, incluzând aici și clasele încuivate, pot avea următorii modificatori de acces: public, protected internal, protected, internal sau private (modificatorul implicit)."

Definițiile claselor, structurilor, interfețelor sau a metodelor pot fi impartite intre doua sau mai multe fisiere sursa:

automat, daca dimensiunea fisierului sursa depaseste 256 KB

prin inserarea cuvântului cheie `partial` in definitia acestora

prin inserarea cuvântului cheie `split` in definitia acestora

Curs, pagina 71: "Definițiile claselor, structurilor, interfețelor sau a metodelor pot fi împărțite între două sau mai multe fișiere sursă cu ajutorul cuvântului cheie `partial`."

Care dintre următoarele afirmații este adevărată?

o proprietate *write-only* va avea numai accesul `get`

o proprietate *write-only* va returna întotdeauna o valoare.

o proprietate poate fi *read-only* sau *write-only*

In documentul "03 Programare (1).pdf", în secțiunea "3.7. Proprietăți", specifică următoarele:

- "La declararea unei proprietăți puteți ignora accesul `set` pentru a obține o proprietate **read-only** sau accesul `get` pentru a obține o proprietate **write-only**". Acest lucru confirmă direct că o proprietate poate fi de oricare dintre aceste două tipuri.

Accesoriul `get` permite citirea valorii, iar accesoriul `set` permite setarea unei valori. Pentru o proprietate *read-write* vom folosi atât `get` cât și `set`, pentru o proprietate *read-only* vom folosi doar `get`, iar pentru *write-only* vom folosi doar `set`.

Un câmp `readonly`:

poate fi initializat in declaratia acestuia sau intr-un constructor al clasei

poate fi initializat numai in declaratia acestuia

poate fi initializat numai intr-un constructor al clasei

Curs, pagina 53: "Atunci când declarația unui câmp include un modificator `readonly`, atribuiriile către câmp nu pot fi făcute decât în declarație sau într-un constructor din aceeași clasă."

Care afirmație este adevărată în legătură cu câmpurile `readonly` și `const`?

valoarea unui câmp `readonly` este determinată la compilare

câmpurile `const` pot avea orice tip de dată dar nu pot fi inițializate cu orice valoare

atât câmpurile `readonly` cât și cele `const` pot fi utilizate în etichetele `case` din blocurile `switch`.

Curs, pagina 53: "Un câmp `const` poate fi initializat numai la declarația acestuia. Un câmp `readonly` poate fi inițializat fie la declarația acestuia, fie în constructor. [...] De asemenea, un câmp `const` reprezintă o constantă la compilare, pe când un câmp `readonly` poate fi utilizat pentru atribuirea de constante la rulare." În cazul `switch`, doar `const` este permis, deoarece valoarea trebuie cunoscută la compilare.

Ce se întâmplă la modificarea valorii unui obiect de tip `StringBuilder`?

se creează un nou obiect de tip `string` care conține valoarea modificată
se creează o copie a obiectului inițial care conține valoarea modificată
se modifică direct valoarea obiectului

Curs, pagina 53: "Ceea ce face deosebită această clasă este faptul că, la apelarea membrilor acestui tip, modificați direct datele obiectului (ceea ce sporește eficiența), nu obțineți o copie a datelor în formatul modificat."

Atributul `System.Flags` aplicat la declarația unei enumerări va permite:

atribuirea către membrii enumerării a unor valori exprimate în format hexazecimal
atribuirea de valori multiple de tipul enumerării prin utilizarea operatorilor binari
atribuirea către membrii enumerării a unor constante definite de API-ul Windows

Curs, pagina 56: "Atributul `Flags` (sau `System.Flags`) aplicat unei enumerări permite atribuirea de valori multiple unui obiect enumerare cu ajutorul operatorilor binari. La întâlnirea acestui atribut compilatorul va permite atribuirea enumerării cu ajutorul operatorilor binari."

Care sunt toate cuvintele cheie folosite pentru implementarea tratării erorilor în C#?

`try, catch, throw, exception`
`try, catch, finally, throw`
`try, catch`
`try, catch, error`

Pentru conversia de la valori `string`, tipurile de date numerice ofera metoda:

`Parse`
`Convert`
`FromString`

Curs, pagina 43: "Conversia valorilor poate fi făcută prin operatorul `cast`, prin metodele `Parse` și `TryParse` ale tipurilor de date numerice, sau prin metodele oferite de clasa `System.Convert` pentru conversia unui tip de bază în alt tip de bază."

Cand convertim un `string` la un numar folosim metoda:

`Int32.Parse(String)`
`Convert.ToInt.Parse(String)`
`Int.Parse(String)`

Curs, pagina 43: "Conversia valorilor poate fi făcută prin operatorul `cast`, prin metodele `Parse` și `TryParse` ale tipurilor de date numerice, sau prin metodele oferite de clasa `System.Convert` pentru conversia unui tip de bază în alt tip de bază."

Valorile `decimal` sunt mai rapide decât valorile `double` atunci când sunt implicate în:

operații de adunare sau scădere
operații de înmulțire sau împărțire
lucrurile cu registrii în virgula mobilă ai procesorului

Curs, pagina 43: "Valorile în virgula mobilă sunt rapide când sunt implicate în operații aritmetice (mai rapide la înmulțire și împărțire decât la adunare și scădere) deoarece aceste operații sunt suportate direct de către componenta hardware."

Narrowing conversion:

apare atunci cand o valoare de un anumit tip este convertita la un tip de marime cel puțin egala

apare atunci cand o valoare de un anumit tip este convertite la un tip de marime mai mica

apare atunci cind o valoare de un anumit tip este convertita la un tip de marime mai mare

Curs, pagina 44: "O conversie de tip *narrowing* apare atunci când o valoare de un anumit tip este convertită la un tip de mărime mai mică."

Care din urmatoarele afirmatii este falsa?

conversiile *widening* se efectueaza intotdeauna fara pierderi de informatii

atunci cand o valoare de un anumit tip este convertita la un alt tip de marime egala putem spune ca apare o conversie de tip *widening*

o conversie *narrowing* apare atunci cand o valoare de un anumit tip este convertita la un tip de marime mai mica

Curs, pagina 44: "Conversiile de tip *widening* apar atunci când o valoare de un anumit tip este convertită la alt tip care este de mărime cel puțin egală. O conversie de tip *narrowing* apare atunci când o valoare de un anumit tip este convertită la un tip de mărime mai mică. [...] Unele conversii *widening* la Single la Double pot provoca o pierdere a preciziei."

O conversie de la `Int32` la `Single` reprezintă:

o conversie *widening* fără pierderi de informații

o conversie *narrowing*

o conversie cu posibile pierderi de informații

Conversia de la `Int32` la `Single` este o **conversie de tip widening**, deoarece domeniul de valori al tipului `Single` (`float`) este mai mare decât cel al lui `Int32` (`int`).

O conversie de la `int` la `float` reprezinta:

o conversie care va genera intotdeauna eroare la compilare

o conversie *narrowing*

o conversie cu posibile pierderi de informații

o conversie cu *widening* fara pierdere de informatii

Conversia de la `int` (care este un alias pentru `System.Int32`) la `float` (alias pentru `System.Single`)

Care dintre următoarele afirmații este adevărată?

fereastra `Command Window` poate apela comenzi ale mediului de dezvoltare integrat Visual Studio

fereastra `Command Window` poate executa secvențe de cod C#

fereastra `Command Window` poate apela orice comandă specifică sistemului de operare Windows

În documentul "01-02 Introducere (1).pdf", secțiunea "Ferestrele `Command` și `Immediate`" de la pagina 28, se specifică următoarele:

- "Fereastra `Command` este folosită pentru a executa comenzi sau alias-uri direct din mediul Visual Studio. Puteți executa atât comenzi de meniu cât și comenzi care nu apar în meniu."

Facilitatile uzuale a unui mediu de dezvoltare sunt:

editarea codului sursa, compilarea, interpretarea, depanarea, testarea, generarea documentatiei
editarea codului sursa, compilarea, interpretarea, depanarea, testarea
editarea codului sursa, compilarea, testarea

Un mediu de dezvoltare este:

o aplicatie pentru scrierea codului unui program
un set de aplicatii care ajuta programatorul in scrierea de programe
o aplicatie pentru depanarea unui program

Cum se face convertirea codului Intermediate Language în cod mașină în timpul unui program C#?

prin lansarea compilatorului Just-In-Time
prin lansarea compilatorului .Net Core specific limbajului C#
prin lansarea compilatorului pentru cod gestionat

Curs, pagina 10: "Deoarece nici un procesor nu poate executa cod IL, CLR trebuie să îl convertească în cod nativ în timpul rulării programului prin lansarea compilatorului JIT și transmiterea către acesta a adresei funcției cu care pornește aplicația."

Utilitarul care permite vizualizare codului Intermediate Language se numeste:

ILDASM
CILDASM
MSILDASM

Curs, pagina 52: "În continuare, dacă vom compila aplicația și vom încărca ansamblul în utilitarul ildasm.exe (acesta permite vizualizarea codului intermediar: MSIL, CIL sau IL) [...]"

Masina Virtuala in care sunt executate programele scrise pentru .NET Framework este cunoscuta sub denumirea:

Microsoft Virtual Machine
Common Language Runtime
Just in Time Machine

Curs, pagina 9: "Programele scrise pentru .NET Framework sunt executate într-o mașină virtuală care gestionează programele în timpul rulării din punct de vedere al necesităților acestora. Această mașină virtuală este cunoscută sub denumirea de Common Language Runtime (CLR) și face parte din .NET Framework."

In Visual Studio .NET o solutie reprezinta:

un grup format din mai multe fisiere care produc la iesire un anumit rezultat
un grup format dintr-unul sau mai multe proiecte care sunt gestionate impreuna
un grup format din mai multe declaratii de functii si variabile care pot rezolva o problema data

Curs, pagina 15: "O soluție reprezintă un grup format din unul sau mai multe proiecte care sunt gestionate împreună."

Un program Visual Basic .NET:

executa numai cod gestionat

poate executa atat cod gestionat cat si cod negestionat

executa cod negestionat atat timp cat nu se precizeaza explicit ca se doreste executarea de cod gestionat

Curs, pagin 11: "Se spune că aplicațiile .NET execută cod gestionat deoarece rulează sub controlul CLR și sunt împiedecate să ruleze cod nesigur care poate afecta rularea sistemului sau poate compromite datele utilizatorilor. [...] Limbajul C# permite scrierea de cod negestionat numai dacă este activată opțiunea corespunzătoare din proprietățile proiectului, prin includerea acestuia într-un bloc de cod unsafe."

O aplicatie C# .NET care utilizeaza pointeri si cod nesigur (unsafe code) va rula:

va fel de rapid ca si aplicatia echivalenta scrisa in Visual Basic

mai lent decat aplicatia echivalenta scrisa in Visual Basic

mai rapid decat aplicatia echivalenta scrisa in Visual Basic

Curs, pagina 10: "O excepție de la această regulă este cazul unei aplicații care utilizează pointeri și cod nesigur (unsafe code). Aceasta poate rula semnificativ mai rapid decât o aplicație echivalentă scrisă în C# sau Visual Basic."

Familia de framework-uri .NET contine:

.NET Framework, .NET Apps

.NET Framework, .NET Core

.NET Framework, .NET Core, Xamarin

Care din urmatoarele afirmatii este falsa?

fereastra Immediate Window poate executa instructiuni C#

fereastra Command Window poate apela comenzi ale mediului de dezvoltare integrat Visual Studio

fereastra Command Window poate apela si comenzi ale sistemului de operare

Curs, pagina 28: "Fereastra Command este folosită pentru a executa comenzi sau alias-uri direct din mediul Visual Studio. Puteți executa atât comenzi de meniu cât și comenzi care nu apar în meniu. [...] Fereastra Immediate este folosită în timpul depanării programelor pentru evaluarea expresiilor, executarea instrucțiunilor, afișarea valorilor unor variabile ș.a.m.d. Aceasta vă permite să introduceți expresii pentru a fi evaluate sau executate în timpul depanării programelor."

Care dintre următoarele afirmații este corectă în legătură cu spațiile de nume din C#?

este permisă existența mai multor clase cu același nume, dacă fiecare face parte dintr-un spațiu de nume diferit

cuvântul cheie `using` indică faptul că programul poate folosi direct numele tipurilor de date definite într-un spațiu de nume, fără a fi nevoie de prefixarea lor cu spațiul de nume

se poate defini un spațiu de nume în interiorul unui alt spațiu de nume

toate afirmațiile de mai sus sunt corecte

niciuna dintre afirmațiile de mai sus nu este corectă

În secțiunea "3.15. Spații de nume", se specifică: "...datorită faptului că numele complet al unei clase conține și denumirea spațiului de nume, puteți defini clase care au același nume, cu condiția ca ele să facă parte din spații de nume diferite."

În secțiunea "3.15.2. Directiva using", se explică: "...dacă un spațiu de nume apare ca argument al directivei using, puteți omite întregul spațiu de nume la declararea unei variabile, făcând astfel codul scris mai compact și mai lizibil".

Secțiunea "3.15.4. Spații de nume încuibate" afirmă: "Așa cum am mai precizat, spațiile de nume pot fi și încuibate". De asemenea, în secțiunea "3.15.3", se menționează că în cadrul unui spațiu de nume se pot declara "alte spații de nume".

Pentru a putea utiliza în cadrul unui fisier sursa un tip de data definit într-un anumit spațiu de nume ve-ți folosi directiva

```
using
imports
include
```

Directiva using permite folosirea tipurilor de date definite într-un alt fisier.

Care proprietăți ale unui formular permit desemnarea acțiunilor implicite efectuate la apăsarea de către utilizator a tastelor `Enter`, respectiv `Esc`?

```
Enter și Escape
AcceptButton și CancelButton
OkCommand și CancelCommand
```

Curs, pagina 137: "Setați pentru formularul în cauză proprietatea `AcceptButton` la butonul care doriți să fie apelat la apăsarea tastei `Enter`, iar proprietatea `CancelButton` la butonul care doriți să fie apelat la apăsarea tastei `Esc`."

Cum se face în C# asocierea prin cod a unui control cu un handler de eveniment?

```
button1.Click += button1_Click;
button1.Click() += button1_Click();
button1.Click = button1_Click();
```

Curs, pagina 92: "Înregistrarea la un anumit eveniment se face prin operatorul `+=` (similar cu adăugarea metodelor în lista de invocare a delegaților multicast)."

La click pe suprafața unui control, evenimentele `MouseDown`, `Click` și `MouseUp` apar în următoarea ordine:

```
Click, MouseDown, MouseUp
MouseDown, Click, MouseUp
MouseDown, MouseUp, Click
```

Curs, pagina 125: "Atunci când dați click pe un control, apar următoarele evenimente în ordinea indicată. Prima instanță a evenimentului `MouseMove` poate apărea de oricâte ori mișcați mouse-ul cât timp butonul de mouse este apăsat. Evenimentul final `MouseMove` apare indiferent dacă mișcați mouse-ul sau nu.

Construirea unei liste de ferestre copil se face in cadrul ferestrei parinte prin setarea pentru controlul principal `MenuStrip` a proprietatii:

`MdiWindowItems`
`MdiChildWindowList`
`MdiWindowListItem`

Curs, pagina 135: "Construirea unei liste de ferestre copil este o sarcină ușoară în Visual C#. Selectați controlul principal `MenuStrip`, iar în fereastra de proprietăți a acestuia setați proprietatea `MdiWindowListItem` la meniul care doriți să conțină lista de ferestre copil. "

Pentru crearea unei aplicati MDI:

se va seta pentru formularul principal proprietatea `IsMdiContainer`

se va seta pentru formularul principal proprietatea `MdiParent`

se va seta pentru formularul principal proprietatea `IsMdiParent` iar pentru celelalte
formulare `IsMdiChild`

Curs, pagina 135: "Pentru a avea o aplicație MDI, creați întâi o aplicație nouă, apoi setați proprietatea `IsMdiContainer` a formularului de start pe `true`."

Curs, pagina 135: "În faza de design, un formular copil MDI are aceeași înfățișare cu a unui formular standard. Pentru a face ca formularul copil să fie afișat în interiorul unui container MDI, trebuie să îi setați acestuia în timpul rulării proprietatea `MdiParent` la containerul MDI."

In mediul Visual Studio.NET puteti accesa elementele copiate anterior in clipboard prin comanda de meniu:

`Last Clipboard Elements`
`Previous Copied Elements`
`Cycle Clipboard Ring`

Curs, pagina 18: "Show Clipboard History (Cycle Clipboard Ring) -- conține ultimele elemente copiate în clipboard. Această comandă copiază în locația curentă din editorul de text elementul curent din clipboard. Prin utilizarea acestei comenzi, puteți parcurge, pe rând, toate elementele copiate în clipboard până la găsirea celui pe care îl doriți.

In Visual Basic .NET cerinta ca toate variabilele sa fie declarate inainte de a fi utilizate se face prin optiunea de compilare:

`Option Strict`
`Option Explicit`
`Option Declare`

Curs, pagina 23: "Option Strict. Atunci când această opțiune este inactivă, Visual Studio permite în cod conversii implicite de la un tip de dată la altul, chiar dacă tipurile de date nu sunt întotdeauna compatibile. "

Curs, pagina 23: "Option Explicit determină din partea Visual Basic cerința ca toate variabilele să fie declarate înainte de a fi utilizate."

In fereastra Immediate Window, cat timp programul este interrupt din depanare, o variabila `x` poate fi evaluata prin comanda:

```
?x  
:x  
x=
```

Curs, pagina 28: "Una dintre cele mai utile comenzi pentru fereastra Immediate este `Debug.Print`. De exemplu, comanda `Debug.Print x` afiseaza valoarea variabilei `x`. Puteti utiliza semnul `?` ca abreviere a acestei comenzi (`123` reprezinta valoarea variabilei `x` afisata in fereastra): `>? X 123`

Comanda `Clean` din meniul build:

- sterge toate rezultatele compilate aplicatiei (EXE sau DLL)
- sterge fisierele de backup asociate proiectului current
- sterge fisierele temporare sau intermediare create la compilare

Curs, pagina 25: "Clean Solution -- aceasta comanda sterge fisierele temporare sau intermediare create la compilarea solutiei, pastrand doar fisierele sursa si rezultatele finale (fisierele EXE sau DLL)."

Pentru afisarea modala a unui formular ii vom apela metoda:

```
DoModal()  
ModalDisplay()  
ShowDialog()
```

Curs, pagina 137: "Metoda `ShowDialog()` are ca rezultat afisarea modala a ferestrei dialog. O fereastră modală menține focalizarea asupra ferestrei, astfel încât utilizatorii nu pot să interacționeze cu alte componente ale aplicației cât timp fereastra modală este deschisă."

Variabilele la nivel de bloc:

- pot fi accesate din afara blocului
- nu pot fi accesate din afara blocului
- pot fi accesate din afara blocului daca acestea sunt de tip `var`

Curs, pagina 37: "Variabilele la nivel de bloc (variabile bloc) pot fi utilizate numai în interiorul blocului în care au fost definite."

In C# optiunea de deducere a tipului unei variabile locale se face:

- automat, prin determinarea tipului valorii care ii este atribuita
- prin utilizarea directivei de compilare `Option Infer`
- prin declararea variabilei ca fiind de tipul implicit `var`

Curs, pagina 24: "[...] în C# poate fi utilizată opțiunea de deducere a unei variabile. Aceasta se aplică însă numai variabilelor locale al nivel de metodă, declarându-le ca fiind de tipul implicit `var`. Compilatorul este cel care determină tipul variabilei în funcție de valoarea care îi este atribuită."

Cum trebuie să fie nivelul de accesibilitate al clasei derivate?

- mai mare sau același cu cel al clasei de bază
- același cu cel al clasei de bază
- mai mic sau același cu cel al clasei de bază

Curs, pagina 83: "Când moșteniți o clasă, nu puteți face clasa derivată mai accesibilă decât clasa ei de bază. De exemplu, dacă moșteniți o clasă internal, nu puteți declara clasa derivată ca public. [...] Această restricție nu se aplică interfețelor pe care le implementați. O clasă public este liberă să implementeze interfețe internal sau private. Totuși, se aplică bazelor unei interfețe: o interfață public nu poate moșteni o interfață internal)."

Dacă doriți ca un membru din clasă derivată să aibă același nume cu cel al unui membru din clasă de bază, fără însă ca acesta să participe la invocarea virtuală, atunci folosiți în declarația membrului clasei derivate:

cuvântul cheie `override`

cuvântul cheie `virtual`

cuvântul cheie `new`

Curs, pagina 86: "Dacă doriți ca membrul din clasă derivată să aibă același nume cu cel al unui membru din clasă de bază, însă nu doriți ca acesta să participe la invocarea virtuală, puteți folosi cuvântul cheie `new`."

Cum poate o clasă derivată să oprească moștenirea caracterului virtual al unui membru din clasă de bază?

prin declararea acestuia ca `sealed override`

prin omiterea cuvântului cheie `virtual` în declarația acestuia

prin omiterea cuvântului cheie `new` în declarația acestuia

Curs, pagina 87: "O clasă derivată poate opri moștenirea caracterului virtual prin declararea unei suprascriseri sigilate. Acest lucru necesită folosirea cuvântului cheie `sealed` înainte de cuvântul `override` în declarația membrului clasei."

Ce proprietate trebuie să setați pentru a desena un text folosind *antialiasing*?

`TextAntialiasing`

`TextSmoothingStyle`

`TextRenderingHint`

Curs, pagina 142: "Puteți seta proprietatea `TextRenderingHint` a unui obiect de tip `Graphics` pentru a-i preciza programului dacă trebuie să utilizeze *antialiasing* pentru reprezentarea mai fină a textului."

Pentru a preciza dacă un text va fi desenat folosind *antialiasing* va trebui să setați proprietatea:

`TextRenderStyle`

`TextSmoothingMode`

`TextRenderingHint`

Curs, pagina 142: "Puteți seta proprietatea `TextRenderingHint` a unui obiect de tip `Graphics` pentru a-i preciza programului dacă trebuie să utilizeze *antialiasing* pentru reprezentarea mai fină a textului."

Pentru umplerea unui obiect `Graphics` cu o culoare dată se va utiliza:

proprietatea `BackgroundColor`

metoda `SetBackgroundColor`

metoda `Clear`

Curs, pagina 143: "`Clear`: Curăță obiectul `Graphics` și îl umple cu o culoare specifică. De exemplu, handler-ul evenimentului `Paint` asociat unui formular poate utiliza sintaxa `e.Graphics.Clear(this.BackColor)` pentru curățarea suprafeței formularului prin utilizarea culorii de fundal."

Clasa `Bitmap` este cuprinsă în spațiul de nume:

`System.Drawing`

`System.Drawing2D`

`System.Graphics`

Curs, pagina 139: "Spațiul de nume `System.Drawing` conține clasele de bază GDI+ și cele mai importante dintre acestea. Din aceste clase fac parte `Graphics`, `Pen`, `Brush`, `Font`, `FontFamily`, `Bitmap`, `Icon` și `Image`."

desenarea de text in interiorul unei suprafete de desenare se face prin apelul metodei:

DrawText
DrawString
RenderText

Curs, pagina 143: "DrawString: Desenează text pe suprafața de desenare."

Obiectele de tip `Brush` determina:

textura si culoarea de umplere a unei suprafete
grosimea si culoarea hasurilor de umplere
culoarea si desenarea de umplere a elementelor grafice

Curs, pagina 145: "Pensulele (Brush) controlează caracteristicile suprafețelor umplute. Ele pot umple o suprafață cu culoare uniformă, hașură, imagine, sau diferite tipuri de gradienti de culoare."

Care dintre următoarele clase permit instanțierea validă a unor obiecte de tip pensulă?

Brush, HatchBrush, PathGradientBrush
Brush, TextureBrush, HatchBrush
SolidBrush, HatchBrush, LinearGradientBrush

Curs, pagina 147: "Clasa Brush însăși este o clasă abstractă, astfel că nu puteți crea instanțe ale clasei Brush. În schimb, puteți crea instanțe ale uneia din clasele derivate *SolidBrush*, *TextureBrush*, *HatchBrush*, *LinearGradientBrush* și *PathGradientBrush*."

Care dintre următoarele afirmații este adevărată?

un obiect de tip `DataAdapter` transferă date între o conexiune și un `DataSet`.
un obiect de tip `DataAdapter` transferă date între un `DataTable` și un `DataSet`
un obiect de tip `DataAdapter` transferă date între o conexiune și un `DataTable`

Curs, pagina 161: "DataAdapter: permit mutarea datelor între baza de date și containerul de date."

Care dintre următoarele afirmații este falsă?

un obiect `DataSet` stochează datele dintr-un tabel al unei baze de date
`BindingSource` încapsulează toate datele din `DataSet` și oferă funcții pentru controlul acestora din cadrul programului
`TableAdapterManager` utilizează relațiile de tip foreign-key pentru determinarea ordinii corecte de trimitere a comenzilor `Insert`, `Update` sau `Delete` către o bază de date

Conform documentului "07 Baze de date (1).pdf", un `DataSet` este o structură mai complexă:

- "Un obiect `DataSet` reprezintă o bază de date."
- "Acesta conține obiecte de tip `DataTable`, care reprezintă tabelele din baza de date."
- "Un `DataSet` poate stoca tabele multiple cu relații complexe părinte-copil și chei unice."

Prin urmare, un `DataSet` poate stoca date din **mai multe tabele**, nu doar dintr-unul singur.

Celelalte două afirmații sunt descrise exact așa în materialul de curs:

BindingSource: "Obiectul `BindingSource` încapsulează toate datele din `DataSet` și oferă funcții pentru controlul acestora din cadrul programului."

TableAdapterManager: "TableAdapterManager utilizează relațiile de tip foreign-key, care relaționează tabelele, pentru determinarea ordinii corecte de trimitere a comenzilor `Insert`, `Update` sau `Delete` de la un `DataSet` către o bază de date..."

Pentru afisarea datei curente si a timpului curent se utilizeaza proprietatile:

`DateTime.Today` **si** `DateTime.Now`

`DateTime.GetDate` **si** `DateTime.GetTime`

`DateTime.CurrentDate` **si** `DateTime.CurrentTime`

Curs, pagina 48: "Structura `DateTime` pune la dispoziția programatorilor o serie de metode și proprietăți utile, cele mai importante dintre acestea putând să le examinați în Tabelul 3.9:

- `Now`: Returnează un obiect `DateTime` setat la data și timpul curent.
- `Today`: Returnează un obiect `DateTime` setat la data curentă."

Funcția `System.Math.Ceiling (-0.5)` **va returna**

-1.0

0.5

0.0

`Math.Ceiling(double value)` returnează cel mai mic întreg mai mare sau egal cu valoarea dată (adică rotunjirea în sus către infinit).

Pentru afisarea tuturor fisierelor existente la calea indicata in operatiile cu fisiere si directoare se foloseste metoda:

`EnumerateDirectories()`

`DirectoriesInfo()`

`EnumerateFiles()`

- `EnumerateFiles()` este metoda folosită pentru a obține o colecție cu toate fișierele dintr-un director specificat;
- `EnumerateDirectories()` returnează doar directoarele, nu fișierele;
- `DirectoriesInfo()` nu este o metodă validă în API-ul pentru fișiere din C#.

In cate tipuri pot fi manipulate proprietatile controalelor?

1

2

3

Proprietățile controalelor în Windows Forms pot fi manipulate în două moduri principale: la *design-time* sau la *run-time*, adică în fereastra designerului sau în cod.

Principalele operatii folosite pentru crearea dinamica a unui control sunt:

instantiere, setare si adaugare

instantiere si adaugare

insatntiere si setare

Crearea dinamică a unui control presupune:

- instanțierea obiectului;
- setarea proprietăților sale;
- adăugarea în interfața grafică (UI).

Pentru stergerea asocierii dintru un control si un handler de evenimente se foloseste:

`Disable()`

`-=`

`Delete()`

Pentru asociere se foloseste `+=`, iar pentru stergere `-=`.